



OSR eHSM Firmware 技术参考手册

版本	1.1
作者	OSR IP
部门	OSR IP
日期	2026 - 06 - 25
分类	用户手册

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

修订历史

版本	修改日期	描述
1.0	2025-01-01	初始版本
1.1	2026-06-25	- 更新 图 3，OTP 分布增加 Patch Cfg、Patch Addr 和 Patch Data 区域。 - 新增 章节 3.5，补充硬件 OTP ROM Patch 机制说明和 OTP Patch 配置表。 - 更新 表 7，OTP 密钥起始偏移从 0x70 调整为 0x118。 - 更新 表 9，补充 OTP Patch 配置表在不同生命周期下的读写限制。 - 更新 表 55 和 表 57，补充 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE 对 Patch 区域的读写限制。

目录

1	概述	11
1.1	系统特性	11
1.2	标准与合规支持	11
2	系统介绍	12
2.1	系统架构	12
2.2	启动与服务流程	13
2.3	错误处理	14
2.3.1	硬件上报错误	14
2.3.2	软件上报错误	14
3	OTP	16
3.1	生命周期	17
3.2	UID	17
3.3	OTP 控制字段	17
3.4	版本计数器	19
3.5	Patch 机制	20
3.5.1	硬件支持	20
3.5.2	OTP Patch 配置表	20
3.6	OTP 密钥排布	22
3.7	OTP 驱动要求	22
3.7.1	OTP 数据读取	22
3.7.2	OTP 数据写入	23
3.7.3	驱动 API	23
4	生命周期	24
4.1	生命周期的定义	24
4.2	不同生命周期的功能支持	24
4.3	修改生命周期	25
5	密码算法	26
5.1	对称加密算法	26
5.2	认证加密(AEAD)算法	26
5.3	哈希摘要算法	26
5.4	消息认证码(MAC)算法	26
5.5	非对称加密算法	27
5.6	密钥交换算法	27
5.7	密钥派生算法	27
5.8	随机数生成	27
5.9	三段式处理	27
5.10	输入数据要求	28
6	密钥管理	30
6.1	密钥类型	30
6.2	密钥权限	30
6.3	通用密钥管理功能	31

6.4	OTP 密钥	32
6.4.1	密钥映射	33
6.4.2	OTP 密钥烧录（安装）	33
7	鉴权	35
7.1	调试鉴权	36
7.2	USER 模式控制功能鉴权	36
8	安全启动	37
8.1	安全启动交互流程	37
8.2	部署说明	37
8.3	镜像格式	38
8.4	验证命令定义	41
8.5	算法配置说明	41
9	安全固件升级	42
9.1	升级镜像制作	42
9.2	升级镜像安装	43
9.3	版本管理	44
9.4	配置说明	44
9.4.1	升级算法配置	44
10	Mailbox 通信协议	45
10.1	硬件说明	45
10.2	协议说明	45
A	Mailbox 命令	48
A.1	全局定义	48
A.2	对称算法	51
A.2.1	symm_cipher	51
A.2.2	aead_gcm	52
A.2.3	aead_ccm	54
A.3	数据摘要算法	55
A.3.1	hash	55
A.3.2	hmac	56
A.4	MAC 算法	58
A.4.1	mac	58
A.5	非对称算法	59
A.5.1	sm2_cipher	59
A.5.2	sm2_sign	60
A.5.3	Rsa_cipher	62
A.5.4	Rsa_sign	62
A.5.5	Ecdsa	64
A.6	随机数生成	65
A.6.1	rng	65
A.7	扩展服务	66
A.7.1	otp_write	66
A.7.2	otp_read	67
A.7.3	get_challenge	67

A.7.4	auth	68
A.7.5	close_debug	68
A.7.6	fw_upgrade	69
A.7.7	soc_verify	70
A.7.8	install_random_key	70
A.7.9	install_encrypt_key	71
A.7.10	get_emu	72
A.7.11	change_lifecycle	73
A.7.12	change_control_field	73
A.7.13	set_uart_baudrate	74
A.7.14	fw_reg_wr	74
A.7.15	fw_reg_rd	75
A.7.16	ehsm_read_ver	76
A.7.17	enter_wfi	76
A.8	密钥管理	77
A.8.1	ehsm_gen_key	77
A.8.2	ehsm_key_derive	77
A.8.3	ehsm_key_exchange	79
A.8.4	ehsm_import_key	80
A.8.5	ehsm_export_key	81
A.8.6	ehsm_remove_key	82
A.8.7	ehsm_get_pub_from_priv	82
B	错误码	84
C	字符调试鉴权协议	88
C.1	获取挑战值指令	88
C.2	签名校验指令	88
C.3	关闭调试指令	89
C.4	测试指令	89
	参考文献	90

图目录

图 1 eHSM 固件系统架构	12
图 2 启动与服务流程	13
图 3 OTP 分布	16
图 4 CPU 读取指令流程	21
图 5 基本鉴权流程	36
图 6 SoC FW 基于 RAM 的部署方式	37
图 7 SoC FW 基于 NVM 的部署方式	38
图 8 安全启动镜像格式示意图	39
图 9 制作升级镜像	42
图 10 MailBox 协议数据流	45
图 11 Mailbox 协议流程	47

表目录

表 1	硬件严重错误	14
表 2	软件错误	15
表 3	FW Control-eHSM 字段说明	18
表 4	FW Control-SOC 字段说明	19
表 5	Version Counter 编码示例	20
表 6	OTP Patch 配置表	21
表 7	OTP 密钥排布表	22
表 8	生命周期定义	24
表 9	生命周期功能限制	24
表 10	不同命令的输入数据要求	28
表 11	各算法对应的分组长度	29
表 12	密钥类型与特性	30
表 13	密钥权限定义	30
表 14	通用密钥管理功能	31
表 15	OTP 密钥重映射示例	33
表 16	鉴权对应密钥	35
表 17	安全启动镜像字段说明	40
表 18	升级镜像字段说明	43
表 19	枚举 process_mode 定义	48
表 20	枚举 mac_direction 定义	48
表 21	枚举 key_type 定义	48
表 22	枚举 sig_direction 定义	48
表 23	枚举 cipher_direction 定义	48
表 24	枚举 msg_type 定义	49
表 25	枚举 auth_challenge_type 定义	49
表 26	枚举 key_algo_id 定义	49
表 27	枚举 usage_permit_flags 定义	50
表 28	结构体 CMD-SYMM-CIPHER 定义	51
表 29	结构体 RSP-SYMM-CIPHER 定义	52
表 30	结构体 CMD-AEAD-GCM 定义	52
表 31	结构体 RSP-AEAD-GCM 定义	54
表 32	结构体 CMD-AEAD-CCM 定义	54
表 33	结构体 RSP-AEAD-CCM 定义	55
表 34	结构体 CMD-HASH 定义	55
表 35	结构体 RSP-HASH 定义	56
表 36	结构体 CMD-HMAC 定义	56
表 37	结构体 RSP-HMAC 定义	57
表 38	结构体 CMD-MAC 定义	58
表 39	结构体 RSP-MAC 定义	59
表 40	结构体 sm2_key 定义	59
表 41	结构体 CMD-SM2-CIPHER 定义	59

表 42	结构体 RSP-SM2-CIPHER 定义	60
表 43	结构体 CMD-SM2-SIGN 定义	60
表 44	结构体 RSP-SM2-SIGN 定义	61
表 45	结构体 rsa_key 定义	61
表 46	结构体 CMD-RSA-CIPHER 定义	62
表 47	结构体 RSP-RSA-CIPHER 定义	62
表 48	结构体 CMD-RSA-SIGN 定义	62
表 49	结构体 RSP-RSA-SIGN 定义	64
表 50	结构体 ecc_key 定义	64
表 51	结构体 CMD-ECDSA 定义	64
表 52	结构体 RSP-ECDSA 定义	65
表 53	结构体 CMD-RNG 定义	65
表 54	结构体 RSP-RNG 定义	66
表 55	结构体 CMD-OTP-WRITE 定义	66
表 56	结构体 RSP-OTP-WRITE 定义	66
表 57	结构体 CMD-OTP-READ 定义	67
表 58	结构体 RSP-OTP-READ 定义	67
表 59	结构体 CMD-GET-CHALLENGE 定义	67
表 60	结构体 RSP-GET-CHALLENGE 定义	67
表 61	结构体 CMD-AUTH 定义	68
表 62	结构体 RSP-AUTH 定义	68
表 63	结构体 CMD-CLOSE-DEBUG 定义	69
表 64	结构体 RSP-CLOSE-DEBUG 定义	69
表 65	结构体 CMD-FW-UPGRADE 定义	69
表 66	结构体 RSP-FW-UPGRADE 定义	69
表 67	结构体 CMD-SOC-VERIFY 定义	70
表 68	结构体 RSP-SOC-VERIFY 定义	70
表 69	结构体 CMD-INSTALL-RANDOM-KEY 定义	70
表 70	结构体 RSP-INSTALL-RANDOM-KEY 定义	71
表 71	结构体 CMD-INSTALL-ENCRYPT-KEY 定义	71
表 72	结构体 RSP-INSTALL-ENCRYPT-KEY 定义	72
表 73	结构体 ehsm_emu_status 定义	72
表 74	结构体 CMD-GET-EMU 定义	72
表 75	结构体 RSP-GET-EMU 定义	73
表 76	结构体 CMD-CHANGE-LIFECYCLE 定义	73
表 77	结构体 RSP-CHANGE-LIFECYCLE 定义	73
表 78	结构体 CMD-CHANGE-CONTROL-FIELD 定义	73
表 79	结构体 RSP-CHANGE-CONTROL-FIELD 定义	74
表 80	结构体 CMD-SET-UART-BAUDRATE 定义	74
表 81	结构体 RSP-SET-UART-BAUDRATE 定义	74
表 82	结构体 CMD-FW-REG-WR 定义	74
表 83	结构体 RSP-FW-REG-WR 定义	75
表 84	结构体 CMD-FW-REG-RD 定义	75
表 85	结构体 RSP-FW-REG-RD 定义	75

表 86	结构体 ehsm_version 定义	75
表 87	结构体 CMD-EHSM-READ-VER 定义	76
表 88	结构体 RSP-EHSM-READ-VER 定义	76
表 89	结构体 CMD-ENTER-WFI 定义	76
表 90	结构体 RSP-ENTER-WFI 定义	77
表 91	结构体 CMD-EHSM-GEN-KEY 定义	77
表 92	结构体 RSP-EHSM-GEN-KEY 定义	77
表 93	结构体 CMD-EHSM-KEY-DERIVE 定义	78
表 94	结构体 RSP-EHSM-KEY-DERIVE 定义	79
表 95	结构体 sm2_param_struct 定义	79
表 96	结构体 CMD-EHSM-KEY-EXCHANGE 定义	79
表 97	结构体 RSP-EHSM-KEY-EXCHANGE 定义	80
表 98	结构体 CMD-EHSM-IMPORT-KEY 定义	80
表 99	结构体 RSP-EHSM-IMPORT-KEY 定义	81
表 100	结构体 CMD-EHSM-EXPORT-KEY 定义	81
表 101	结构体 RSP-EHSM-EXPORT-KEY 定义	82
表 102	结构体 CMD-EHSM-REMOVE-KEY 定义	82
表 103	结构体 RSP-EHSM-REMOVE-KEY 定义	82
表 104	结构体 CMD-EHSM-GET-PUB-FROM-PRIV 定义	82
表 105	结构体 RSP-EHSM-GET-PUB-FROM-PRIV 定义	83
表 106	错误码定义	84

1 概述

本文档描述 OSR eHSM Firmware 相关的功能和技术要点。

1.1 系统特性

OSR eHSM Firmware 是一个基于 RISC-V 架构的嵌入式硬件安全模块固件，具有以下核心特性：

- **硬件安全保障：**基于专用安全硬件，提供密钥隔离、防篡改和物理攻击检测
- **完整密码算法支持：**涵盖对称/非对称/哈希/MAC 等多类算法，支持国密和国际标准
- **密钥管理支持：**安全的密钥生成、导入导出、派生协商等功能
- **安全启动与升级：**提供完整的信任链验证和防回滚保护，抗注入攻击
- **灵活的生命周期管理：**支持开发、制造、用户、调试、销毁等多种安全状态

1.2 标准与合规支持

eHSM Firmware 遵循多项国际和国内安全标准：

- **国密算法标准：**全面支持 SM2/SM3/SM4 等国密算法
- **国际密码标准：**兼容 AES、RSA、ECDSA、SHA 等国际算法
- **代码安全规范：**遵循 MISRA-C 2012 编码标准，确保代码质量

2 系统介绍

本章节描述 eHSM Firmware 的系统架构、启动流程、命令交互机制和错误处理策略，帮助用户理解系统的运行原理和工作方式。

2.1 系统架构

eHSM Firmware 采用分层架构设计：

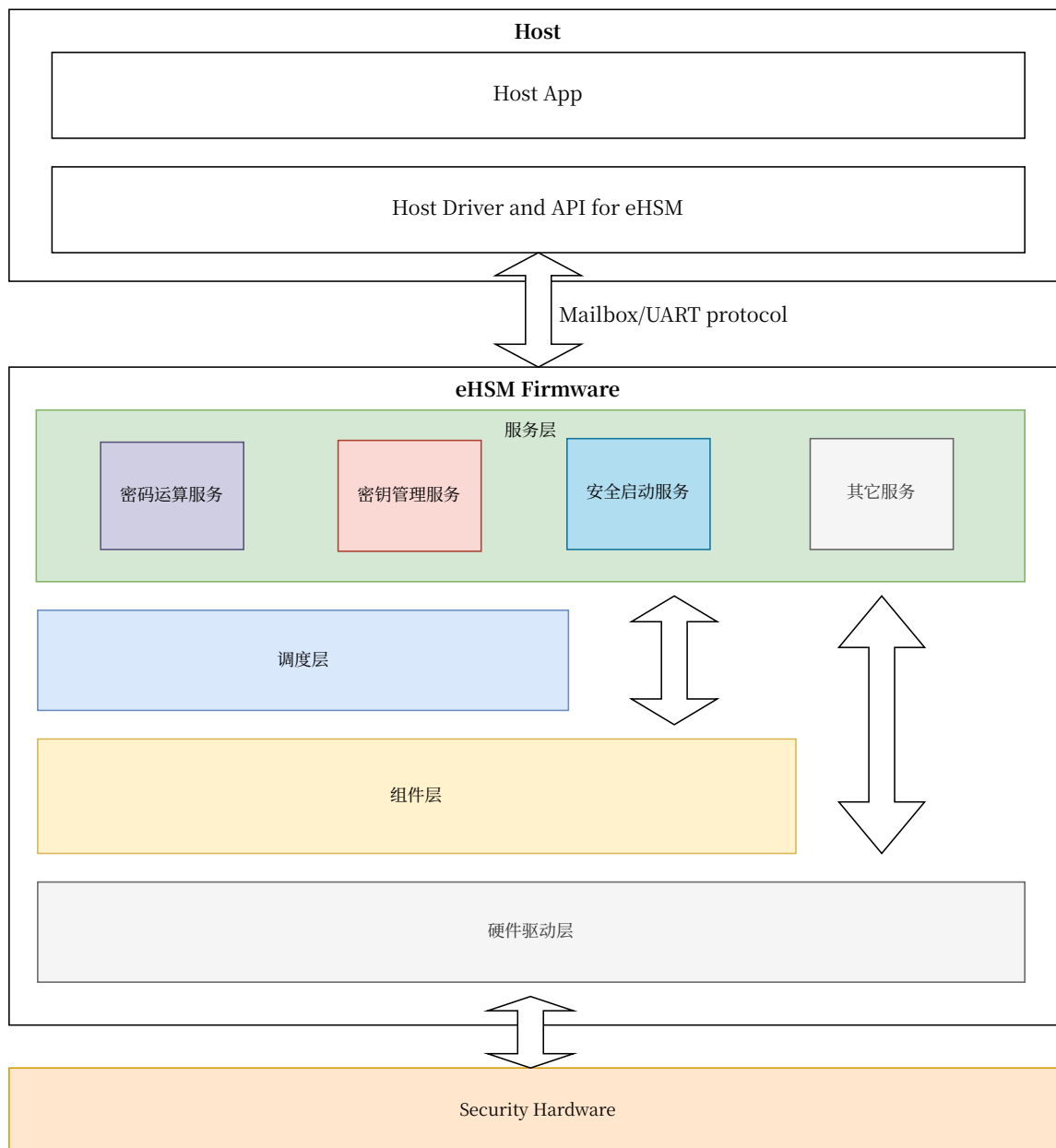


图 1 eHSM 固件系统架构

系统主要由以下几个层次组成：

- **调度层：**负责任务调度和命令分发，以及命令的预处理

- **组件层**：提供软件功能组件，如通信、存储管理等
- **硬件驱动层**：封装硬件操作的驱动模块
- **服务层**：对外提供功能服务的模块
 - **密码运算服务**：支持各种密码算法运算功能
 - **密钥管理服务**：支持密钥的生成、派生、协商、导入、导出、删除功能
 - **安全启动服务**：支持安全启动镜像校验、升级镜像等功能
 - **其它服务**：其它系统相关服务

2.2 启动与服务流程

eHSM Firmware 的启动与服务流程如下图所示：

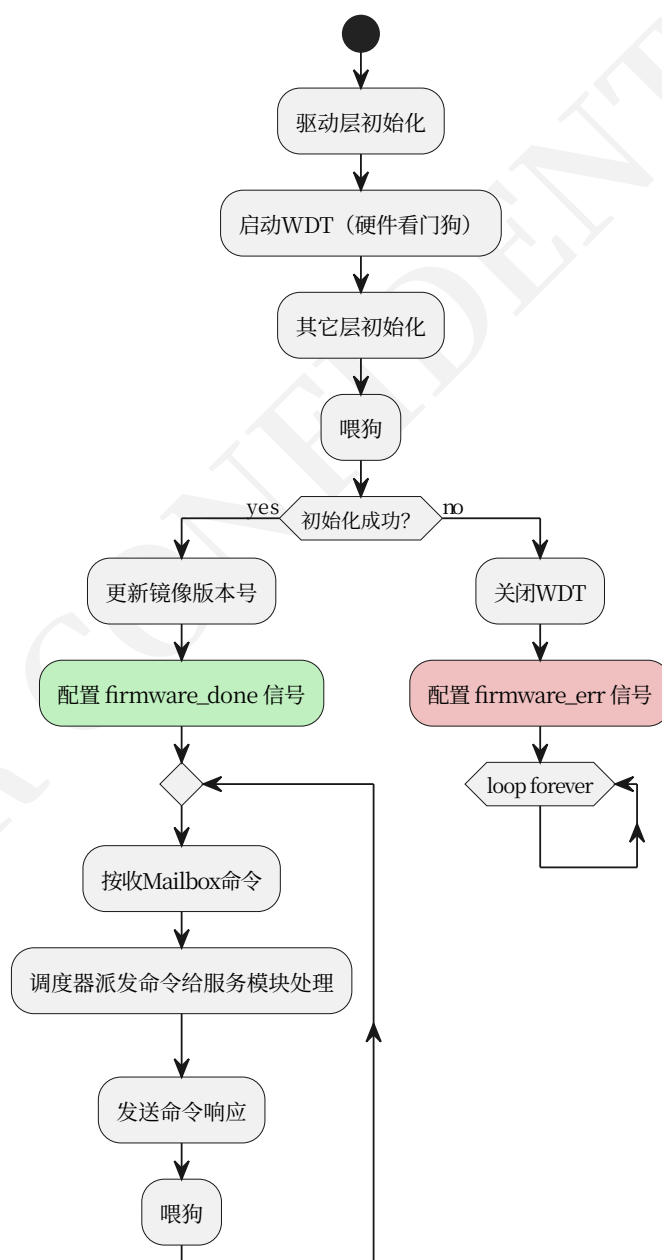


图 2 启动与服务流程

- 作为从机设备，eHSM 仅被动接收并处理 Mailbox 命令，不会主动发起操作
- Firmware 会在多个 Mailbox channel 上轮流处理输入的命令，避免在一个 channel 上一直处理命令而导致其它 channel 无法服务的情况出现
- WDT 的超时时长为 200ms¹，在使用密码算法等命令时，需要根据实际性能确定最大输入数据长度，避免处理时间过长导致命令超时
- 当系统遇到严重错误时，Firmware 会配置错误信号通知 SOC 端，然后 eHSM 进入挂起状态，不再响应 Mailbox 命令
- Bootloader 在校验 eHSM/SOC 启动镜像时，如果有版本更新，会将新的版本号写入 DRAM 并传递给 Firmware，由 Firmware 来更新 OTP 中的版本数据，以避免在 Bootloader 下进行 OTP 写入操作。关于镜像的版本号，请参考 [章节 3.4](#)。

2.3 错误处理

eHSM Firmware 报告的错误分成 2 类：

1. **命令处理错误**：这种错误由 Mailbox 响应返回，表明在命令的处理过程中遇到了错误或问题，但不影响 eHSM 继续运行和提供服务。关于命令错误码的定义，请参考 [表 106](#)。
2. **EMU 上报错误**：这种错误由 EMU 的错误寄存器直接送到 SOC 端，严重的错误需要将 eHSM 重新复位才可能解决问题。

EMU 上报错误分为硬件报告的错误和软件（Firmware）报告的错误。

2.3.1 硬件上报错误

硬件报告的错误可参考 [\[1\]](#) 中“o_hsm_err_hw”表格以及 EMU 模块的 `ERR_HW_0` 和 `ERR_HW_1` 寄存器说明。Firmware 会捕获 [表 1](#) 中的错误并进入挂起状态，其它硬件错误 Firmware 会自动处理或忽略，不影响 Firmware 继续进行服务，但硬件仍然会通过 EMU 直接上报 SOC。

表 1 硬件严重错误

错误位	描述
62:57	DMAC Timeout Error
55:46	Bus Parity Error
40	看门狗计时器超时
35	OTP 密钥 CRC 错误
29:24	SOC Bus Error
19:11	Memory ECC Multi-Bits Error

2.3.2 软件上报错误

软件会通过硬件的 EMU 寄存器报告自己的错误，使用到了 EMU 模块的 `ERR_FW_0` 和 `ERR_FW_1` 寄存器上报 SOC。

软件报告的错误实际上区分了级别，不但有错误，也有警告，定义如下：

1. **警告**：仅上报 SOC，Firmware 继续执行

¹极少数命令在处理时会临时关闭 WDT，例如生成 RSA 密钥，因为处理时间较长且随机

2. 错误：除了上报 SOC，Fireware 会挂起并拒绝继续服务

软件错误定义与处理方式如下表所示：

表 2 软件错误

错误位	错误描述	处理方式
48	OTP 写入失败	挂起
47	检测到错误注入攻击	挂起
45	硬件寄存器写入失败	挂起
37	WDT 中断触发 ²	挂起
35	UART 调试鉴权协议错误	继续执行
34	Mailbox 调试鉴权协议错误	继续执行
33	清除 s2h_note 寄存器未成功	继续执行
32	Mailbox 中断触发但 s2h_note 寄存器未置 1	继续执行
31:28	CPU 异常 ³	挂起

²这是软件捕获的中断并上报的信号，和硬件报告的不是同一个信号，但来源都是 WDT

³这 4 位包含 RISC-V CPU 的 mcause 寄存器的底 4 位，代表引起错误的异常代码，可参考 [这里](#)

3 OTP

OPT 数据用于对 eHSM 的功能进行配置, OTP 的数据分布如下图:

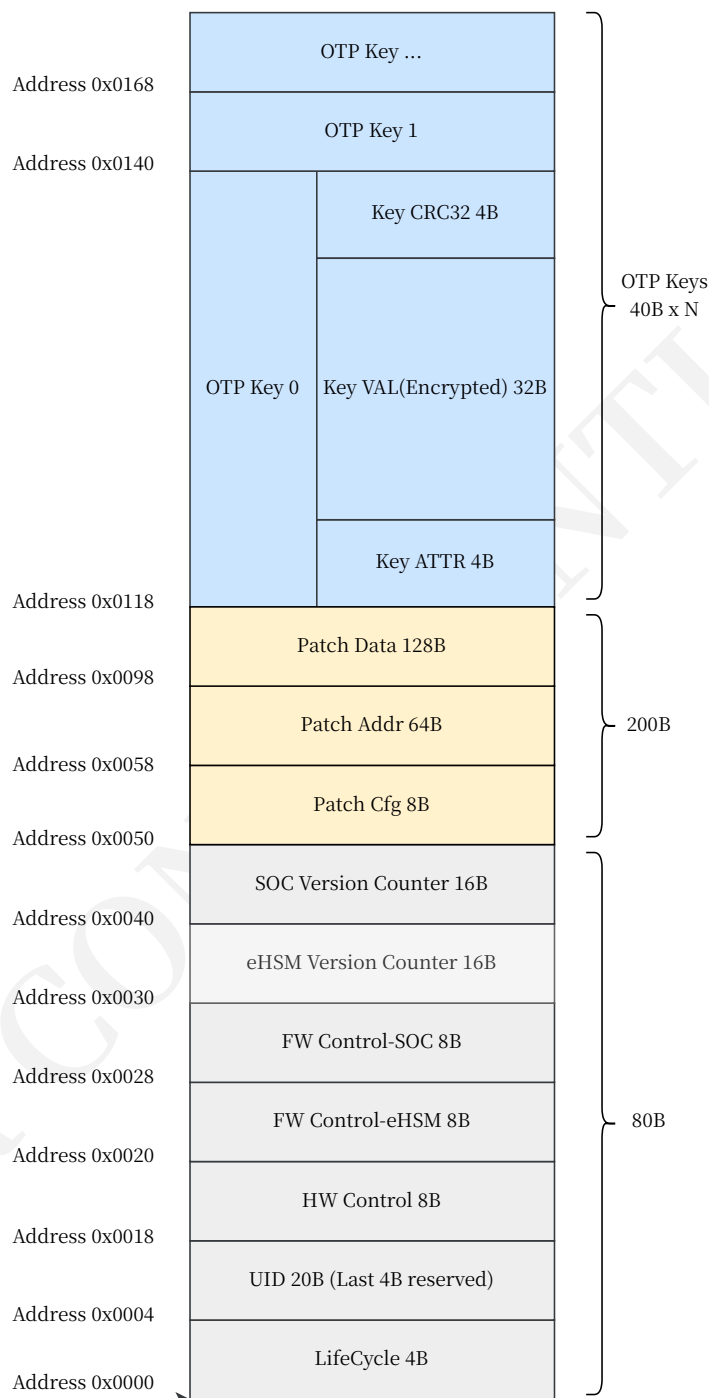


图 3 OTP 分布

其中:

- 生命周期数据(LifeCycle) 由硬件读取, 如果值不正确, eHSM 的 CPU 会被挂起无法运行
- UID 字段共 20 字节, 其中前 16 字节为有效的 UID 数据, 后 4 字节为 CRC32 校验值。

- 控制数据会在上电时被硬件加载到系统寄存器中，软件直接从系统寄存器获取相应的配置值
- Patch 配置表数据（使能后）会在上电时被写入到硬件 Patch 模块
- 密钥数据会在上电时被硬件解密到 KMU 中
- 具体细节请参考 [1] 的相关章节。

3.1 生命周期

请参考 [章节 4](#)。

3.2 UID

UID 字段总共占用 20 字节，其中前 16 字节为 UID 数据，后 4 字节为前 16 字节的 CRC32 校验。生产时应确保每个 eHSM 写入的 16 字节 UID 是唯一的，否则一些鉴权操作会存在安全风险。

其中 CRC32 算法的多项式为：0x04C11DB7，初始值为：0xFFFFFFFF。

UID 由 Firmware 读取并使用。

3.3 OTP 控制字段

注：

- 本节描述的控制字段在 OTP 中都以小端序存储；
- 硬件控制字段请参考 [1]。

表 3 FW Control-eHSM 字段说明

Bit.idx	Name	Description
63:40	Reserved	
39:36	EhsmCodeUpgradeAlg	eHSM 安全升级镜像验签算法选择: • 0b0000: SHA256-RSA2048-PSS • 0b0001: SM3-SM2 • 0b0100: AES128-CMAC • 0b0101: SM4-CMAC • 0b0110: SHA256-RSA3072-PSS • 0b0111: SHA256-ECDSA-SECP256R1 • Other: 保留
35:32	EhsmCodeVerifyAlg	eHSM 安全启动镜像验签算法选择: • 0b0000: SHA256-RSA2048-PSS • 0b0001: SM3-SM2 • 0b0010: AES128-CMAC • 0b0011: SM4-CMAC • 0b0110: SHA256-RSA3072-PSS • 0b0111: SHA256-ECDSA-SECP256R1 • Other: 保留
31:0	Reserved	

表 4 FW Control-SOC 字段说明

Bit.idx	Name	Description
63:8	Reserved	
7:4	SocUpgradeAlg	SOC 安全升级镜像算法选择: • 0b0000: SHA256-RSA2048-PSS • 0b0001: SM3-SM2 • 0b0100: AES128-CMAC • 0b0101: SM4-CMAC • 0b0110: SHA256-RSA3072-PSS • 0b0111: SHA256-ECDSA-SECP256R1 • Other: 保留
3:0	SocBootAlg	SOC 安全启动镜像密钥算法选择: • 0b0000: SHA256-RSA2048-PSS • 0b0001: SM3-SM2 • 0b0010: AES128-CMAC • 0b0011: SM4-CMAC • 0b0110: SHA256-RSA3072-PSS • 0b0111: SHA256-ECDSA-SECP256R1 • Other: 保留

3.4 版本计数器

OTP 中包含了 eHSM FW 和 SOC FW 的版本计数器 (Version Counter)。Version Counter 是一个 16 字节的数据, 用于管理固件的版本升级, 基于 OTP 单向修改的特性, 对这个数有一定的要求。

假设 OTP 的默认值为 0, 只能修改为 1, 则:

- 16 字节的数据作为一个小端序的大数
- 大数从 0 开始向上增加, 每次从最低的为 0 的位变为 1
- 升级时 Version Counter 必须大于等于 OTP 中记录的 Version Counter 才能成功
- 总共支持 128 个版本

例:

表 5 Version Counter 编码示例

版本	大数值	OTP 存储值
1	0x00000000000000000000000000000001	01000000000000000000000000000000
2	0x00000000000000000000000000000003	03000000000000000000000000000000
15	0x000000000000000000000000000007FF	FF7F0000000000000000000000000000
64	0x0000000000000000FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF0000000000000000
128	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF

3.5 Patch 机制

3.5.1 硬件支持

硬件的 IPatch 模块支持以下特性：

- 将上电过程中硬件 Boot 模块从 OTP 读取的 Patch 数据保存在 IPatch 模块内
- 将 CPU 对 IBUS 上指定地址读取的 32 位指令替换为另一个 32 位指令或两个 16 位指令，替换范围为 0x10000000 - 0x1003FFFF
- 支持 32 个 Patch 入口，Patch 的地址必须 32 位对齐

注：

- 硬件控制字段包含一个 2 比特的 IPatch 使能总开关，只有这个总开关配置为使能硬件 IPatch 功能，OTP Patch 配置表才会生效。请参考 [1]。
- IPatch 模块仅支持对 32 位对齐的地址打补丁，因此，如果替换 32 位指令但地址不是 32 位对齐时，需要消耗 2 个补丁点来替换
- 由于 IPatch 本质上是对 ROM 数据进行替换，因此理论上可以将任何有效范围内的地址的指令替换为其它指令。

3.5.2 OTP Patch 配置表

OTP 包含 Patch 配置表，共支持 32 个 Patch 行，可以实现最多 32 个 words 的指令数据替换，替换的地址必须 word 对齐。对于 $0 \leq i \leq 31$ ，每个 Patch 行由一个 2 比特 Patch Cfg[2i+1:2i]，一个 16 比特 Patch Addr[16i+15:16i]，以及一个 32 比特 Patch Data[32i+31:32i] 组成。

以下是 OTP Patch 配置表：

表 6 OTP Patch 配置表

参数	OTP 偏移(字节)	说明
Patch Cfg	0x50	共 8 字节, 包含 32 个 Patch 行使能, 每 2 比特对应一个 Patch 行的使能开关, 对于 $0 \leq i \leq 31$ - 当 $\text{Patch Cfg}[2i+1:2i] = 2b00$ 或 $2b11$ 时, 该 Patch 行无效 - 当 $\text{Patch Cfg}[2i+1:2i] = 2b01$ 或 $2b10$ 时, 该 Patch 行生效
Patch Addr	0x58	共 64 字节, 包含 32 个地址偏移, 每 16 比特对应一个被替换的 IROM 地址偏移, 对于 $0 \leq i \leq 31$, 实际对应被替换的 IROM 地址计算规则为: 替换的 IROM 地址 = IROM 起始地址 + $(\text{Patch Addr}[16i+15:16i] \ll 2)$ 例如若 IROM 起始地址为 $0x10000000$, 第 0 个 $\text{Patch Addr}[15:0] = 0x0010$ 时, 则第 0 个 Patch 行实际被替换的 IROM 地址为 $0x10000000 + (0x0010 \ll 2) = 0x10000040$ 。
Patch Data	0x98	共 128 字节, 包含 32 个 words 待替换的指令数据, 对于 $0 \leq i \leq 31$, $\text{Patch Data}[32i+31:32i]$ 代表将要写入 $\text{Patch Addr}[16i+15:16i]$ 指向的 IROM 地址的数据。

注:

- 当硬件控制字段配置 IPatch 模块使能 (请参考 [1]), 且 OTP Patch Cfg 配置开启, 同时对应的 OTP Patch Addr 和 Patch Data 有效时, 才会进行指令数据的替换。

下图是 CPU 读取 IROM 指令的流程:

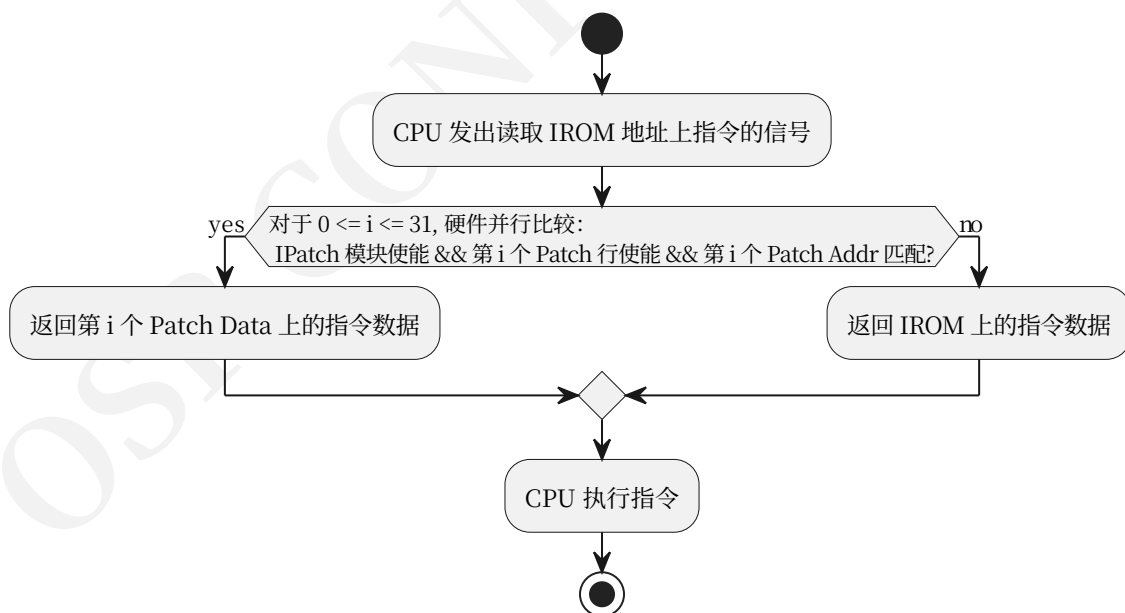


图 4 CPU 读取指令流程

⁴本表中 i 指比特序号, 例如对于 Patch Cfg, 当 $i = 0$ 表示 Patch Cfg[1:0] 比特; $i = 31$ 表示 Patch Cfg[63:62] 比特。其他参数中的 i 同理。

3.6 OTP 密钥排布

OTP 密钥存储在 OTP 偏移 0x118 开始的位置，以下为 OTP 的密钥排布（支持定制位置及复用）：

表 7 OTP 密钥排布表

ID	Level	OTP 偏移(字节)	密钥用途
0	-	280	Chip Root Key
1	-	320	Device Root Key
2	-	360	User Root Key
3	1	400	eHSM Debug Verify Key
4	1	440	eHSM FW Verify Key
5	1	480	eHSM Encrypt Key
6	1	520	eHSM Upgrade Encrypt Key
7	1	560	eHSM Upgrade Verify Key
8	1	600	eHSM Private Key
9	2	640	Soc Debug Verify Key
10	2	680	Soc FW Verify Key
11	2	720	Soc Encrypt Key
12	2	760	Soc Upgrade Encrypt Key
13	2	800	Soc Upgrade Verify Key
14	2	840	Soc Private Key
15	2	880	Secret Key
16	2	920	User Auth Key

注：

- Level 1 的密钥使用 Chip Root Key 加密
- Level 2 的密钥使用 Device Root Key 加密
- Key1 (Device Root Key) 和 Key2 使用 Chip Root Key 加密，忽略 Level 配置
- Key0 (Chip Root Key) 使用 RTL KEY 加密，忽略 Level 配置

相应密钥说明可参考 [\[1\]](#) 的对应章节。密钥支持映射以节省 OTP 空间，请参考 [章节 6.4.1](#)。

3.7 OTP 驱动要求

3.7.1 OTP 数据读取

eHSM Firmware 在运行过程中需要读取 OTP 数据。通常情况下，eHSM 硬件设计已确保可通过总线直接读取 OTP 数据，因此无需特定驱动程序。若存在特殊情况，客户应负责实现读取 OTP 数据的驱动程序，并与 Firmware 代码集成。

3.7.2 OTP 数据写入

eHSM Firmware 在运行过程中需求支持生命周期切换、OTP 密钥安装、镜像版本号更新等功能，需要向 OTP 写入数据。客户应负责实现 OTP 写入驱动，并与 Firmware 代码集成，以满足相应功能要求。

3.7.3 驱动 API

请参考 [\[2\]](#)。

4 生命周期

4.1 生命周期的定义

eHSM 支持以下生命周期⁵:

表 8 生命周期定义

生命周期	简称	OTP 值	Owner	说明
TEST_MODE	TEST	0x00000000 ⁶	Chip Vendor	芯片测试
DEVELOP_MODE	DEV	0x42818414	Chip Vendor	固件开发
MANUFACTURE_MODE	MANU	0x46C1A416	Device Vendor	设备开发/测试/制造
USER_MODE	USER	0x57C1EC96	Customer	用户使用设备
DEBUG_MODE	DEBUG	0xD7C5FCDE	Chip Vendor	芯片/设备调试
DESTROY_MODE	DESTROY	0xFFFFFFFF	-	芯片/设备不可用
UNDEFINE_MODE	UNDEF	其它值	-	等同于 DESTROY_MODE

4.2 不同生命周期的功能支持

eHSM Firmware 在不同生命周期下的功能及限制如下表所示:

表 9 生命周期功能限制

功能	TEST	DEV	MENU	USER	DEBUG
获取版本信息	支持	支持	支持	支持	支持
开启 eHSM 调试	硬件默认开启	硬件默认开启	硬件默认关闭, 软件鉴权开启	硬件默认关闭, 软件不支持打开	硬件默认关闭, 软件鉴权开启
关闭 eHSM 调试	软件可关闭, 但硬件实际无法关闭	软件可关闭	软件可关闭	N/A	软件可关闭
开启 SOC 调试	硬件默认开启	硬件默认开启	硬件默认开启	硬件默认关闭, 软件鉴权开启	硬件默认关闭, 软件鉴权开启
关闭 SOC 调试	软件可操作, 但硬件实际无法关闭	软件可关闭	软件可关闭	软件可关闭	软件可关闭
切换生命周期	支持	支持	支持	必须鉴权后切换 ⁷	支持
镜像升级	支持	支持	支持	支持	支持
校验 SOC 启动镜像	支持	支持	支持	支持	支持
校验 naked 镜像	支持	支持	不支持	不支持	不支持

⁵可参考 [1] 的相关章节

⁶以小端存储, 下同

⁷USER_AUTH 鉴权用于切换生命周期的授权

功能	TEST	DEV	MENU	USER	DEBUG
修改 OTP HW 控制 字段	支持	支持	不支持	不支持	不支持
修改 OTP eHSM 控制 字段	支持	支持	不支持	不支持	不支持
修改 OTP SOC 控制 字段	支持	支持	支持	不支持	不支持
修改 OTP Patch 配置表 ⁸	支持	支持	不支持	不支持	不支持
OTP 写	支持	支持	仅支持写 OTP 扩展区域 ⁹	不支持	不支持
OTP 读	支持	支持	支持	不支持	不支持
CFG 读写	支持	支持	不支持	不支持	不支持
OTP 密钥安装	支持	支持	仅支持安装 Level2 密钥	不支持	不支持

4.3 修改生命周期

请使用 [切换生命周期 76](#) 命令进行生命周期的切换。注意，在 USER 模式下时，必须先进行鉴权，才可以切换生命周期，见 [章节 7](#)。

⁸OTP Patch 配置表（详见 [表 6](#)）仅支持通过 OTP 读写指令进行读写

⁹OTP 中密钥区域之后的区域，如果此区域存在

5 密码算法

eHSM Firmware 提供完整的密码算法支持，涵盖对称加密、非对称加密、哈希摘要、消息认证码、认证加密等多个类别。

5.1 对称加密算法

支持多种对称加密算法和工作模式：

- [AES 算法 28](#) - 支持 AES 128/192/256 位密钥
- [SM4 算法 28](#) - 国密对称算法
- [3DES 算法 28](#) - 支持 128/192 位密钥
- [DES 算法 28](#) - 标准 DES 加密

支持以下工作模式：

- ECB
- CBC
- OFB
- CFB
- CTR
- XTS

5.2 认证加密(AEAD)算法

提供加密和认证一体化的算法：

- [AEAD-GCM 算法 30](#) - AES-GCM 和 SM4-GCM 模式
- [AEAD-CCM 算法 32](#) - AES-CCM 和 SM4-CCM 模式

5.3 哈希摘要算法

支持多种哈希算法：

- [SHA 算法 34](#) - SHA-1、SHA-224、SHA-256、SHA-384、SHA-512、SHA-512/224、SHA-512/256
- [SHA-3 算法 34](#) - SHA3-224、SHA3-256、SHA3-384、SHA3-512
- [SM3 算法 34](#) - 国密哈希算法
- [MD5 算法 34](#) - MD5 摘要算法

5.4 消息认证码(MAC)算法

提供消息完整性验证：

- [HMAC 算法 36](#) - 基于各种哈希算法的 HMAC

- [CMAC 算法 38](#) - AES-CMAC 和 SM4-CMAC
- [CBC-MAC 算法 38](#) - CBC 模式消息认证码
- [GMAC 算法 38](#) - GCM 模式消息认证码

5.5 非对称加密算法

支持多种公钥密码算法：

- [RSA 加解密算法 46](#) - 支持 1024/2048/3072/4096 位密钥
- [RSA 签名验签算法 48](#) - 支持无填充和 PSS 填充
- [SM2 加解密算法 41](#) - SM2 国密椭圆曲线加密
- [SM2 签名验签算法 43](#) - SM2 国密椭圆曲线签名
- [ECDSA 签名验签算法 51](#) - 支持多种标准椭圆曲线
- [Ed25519 算法 51](#) - 爱德华曲线数字签名

5.6 密钥交换算法

支持密钥协商算法：

- DH 算法 - Diffie-Hellman 密钥交换
- ECDH 算法 - 椭圆曲线 Diffie-Hellman
- X25519 算法 - 蒙哥马利曲线密钥交换
- SM2 密钥交换 - 基于 SM2 的密钥协商

密钥交换命令见 [密钥交换命令 96](#)。

5.7 密钥派生算法

支持的密钥派生算法：

- KDF-X963: ANSI X9.63 密钥派生算法
- PBKDF2: PKCS#5 v2.0 密钥派生算法

5.8 随机数生成

- [随机数生成算法 53](#) - 基于硬件 TRNG 的真随机数生成

5.9 三段式处理

大部分算法支持三段式处理，即 START、UPDATE、FINAL 三个阶段，可用于大量数据的分次计算¹⁰。如果一次计算的数据量较小，建议使用 ONE-PASS 模式一次性计算完成以提高性能。见：[表 19](#)。

¹⁰还有一个 STREAMSTART 阶段，实际是 START 和 UPDATE 的组合。

5.10 输入数据要求

在使用密码算法的命令时，对输入数据有一定的要求，在以下描述中，数据**对齐**表示数据长度是对应算法分组长度的整数倍。

以下是几种数据对齐要求的定义：

- **严格对齐**：算法的特性或规范要求分组对齐，否则会报错
- **优先对齐**：对齐的需求源于 DMA 的使用；若数据对齐，则性能会比非对齐的处理过程要好
- **不作要求**：对数据对齐不作要求；即便数据不对齐，也不会影响过程及性能

不同命令对输入数据的要求见 [表 10](#)。

表 10 不同命令的输入数据要求

算法分类	阶段				其它要求或说明
	START	UPDATE	FINAL	ONE-PASS	
对称加密（有填充）	-	严格对齐	不作要求	不作要求	1. XTS 模式要求 ONE-PASS 及 FINAL 接口至少传入 2 个分组数据
对称解密（有填充）	-	严格对齐	严格对齐	严格对齐	1. FINAL 必须传入数据 2. XTS 模式要求 ONE-PASS 及 FINAL 接口至少传入 2 个分组数据
对称加解密（无填充）	-	严格对齐	严格对齐	严格对齐	1. XTS 模式要求 ONE-PASS 及 FINAL 接口至少传入 2 个分组数据
对称 MAC ¹¹	-	优先对齐	优先对齐	不作要求	1. CMAC 的 FINAL 必须输入数据 2. CBC-MAC 的总输入长度不能为 0
AEAD	-	严格对齐	不作要求	不作要求	1. CCM 的 FINAL 必须输入数据
HASH/HMAC	-	优先对齐	优先对齐	不作要求	-
非对称签名验签	-	优先对齐	优先对齐	不作要求	1. 由于 SM2 会先自动计算 32 字节 Za 并 UPDATE, 因此 SM2 第一次 UPDATE 长度应当为分组大小整数倍+32 字节 2. 消息数据实际是输入到 HASH 算法，分组长度指使用的 HASH 算法的分组长度
非对称加解密	-	-	-	不作要求	1. 非对称加解密不支持三段式 2. SM2 的输入不能超过 1K 3. RSA 的输入不能超过 N 的长度

[表 11](#) 列举了各算法的分组长度：

¹¹包括 CMAC、CBC-MAC、GMAC 算法

表 11 各算法对应的分组长度

算法	分组长度 (字节)
AES、SM4	16
DES、3DES	8
MD5、SHA-1、SHA-224、SHA-256、SM3	64
SHA384、SHA512、SHA-512/224、SHA-512/256	128
SHA3-224	144
SHA3-256	136
SHA3-384	104
SHA3-512	72

6 密钥管理

本章描述 eHSM Firmware 的密钥管理机制，简单概括如下：

- 支持 2 种密钥：OTP 密钥以及通用密钥
- 所有密钥通过一个 32 位的整数 KEY_HANDLE 来引用
- 所有密钥绑定一组权限属性，权限会限制密钥的访问和使用
- 密码算法功能通过在 Mailbox 命令中指定 KEY_HANDLE 来使用相应的密钥

6.1 密钥类型

不同密钥类型的相关特性如下表：

表 12 密钥类型与特性

特性	OTP 密钥	通用密钥
KEY_HANDLE 值	0x002xxxxx	0x001xxxxx
存储位置	OTP	DRAM
作为传输密钥	可以（特定 OTP 密钥）	可以（从特定 OTP 密钥派生）
算法是否可引用	可以（特定 OTP 密钥 + 权限）	可以（权限）

注：

- 除 OTP 密钥外，通用密钥在 eHSM 复位后会丢失
- 通用密钥支持单个密钥导入/导出

6.2 密钥权限

表 13 密钥权限定义

权限名	值	说明
KEY_USAGE_SIGN	0x1	密钥可以用于签名，或生成 MAC
KEY_USAGE_VERIFY	0x2	密钥可以用于验证或验证 MAC
KEY_USAGE_ENCRYPT	0x4	密钥可以用于加密
KEY_USAGE_DECRYPT	0x8	密钥可以用于解密
KEY_USAGE_KEYCREATION	0x80	密钥可以用于派生或者协商出新密钥
KEY_USAGE_CREATION_TRANSP_KEY	0x100	当 KEY_USAGE_KEYCREATION 权限已设置，此权限： • 设置：密钥可以用于创建（派生）传输密钥 • 未设置：密钥可以用于创建（派生）非传输密钥
KEY_USAGE_TRANSPORT	0x400	密钥可以用于传输（导入/导出）时保护密钥（加密或 MAC）
KEY_PERMIT_REMOVE	0x800	密钥允许删除
KEY_PERMIT_IMPORT_PLAINTEXT	0x1000	密钥允许明文导入

权限名	值	说明
KEY_PERMIT_IMPORT_CIPHERTEXT	0x2000	密钥允许密文导入
KEY_PERMIT_EXPORT_PLAINTEXT	0x4000	密钥允许明文导出
KEY_PERMIT_EXPORT_CIPHERTEXT	0x8000	密钥允许密文导出
KEY_PERMIT_WR_PRT	0x80000	密钥不允许更新（写保护）

基于安全考虑，以下是密钥权限属性之间的额外约束：

1. 密钥拥有 KEY_USAGE_TRANSPORT 权限，则禁止拥有其它 KEY_USAGE_XXX 权限；
2. 密钥拥有 KEY_USAGE_TRANSPORT 权限，则禁止拥有导出权限；
3. 密钥拥有 KEY_USAGE_CREATION_TRANSP_KEY 权限，则必须先拥有 KEY_USAGE_KEYCREATION 权限；
4. 仅 OTP 密钥可以拥有 KEY_USAGE_CREATION_TRANSP_KEY 权限。
5. 仅 OTP 密钥或从 OTP 派生的密钥可以拥有 KEY_USAGE_TRANSPORT 权限。

6.3 通用密钥管理功能

支持的通用密钥管理功能：

表 14 通用密钥管理功能

功能	说明	输入	输出
密钥生成 91	生成一个随机密钥/密钥对	• 算法类型 • 权限 • KEY_HANDLE	• KEY_HANDLE
密钥派生 93	从指定密钥派生一个密钥	• 根密钥 KEY_HANDLE • 算法类型 • 权限 • 派生参数 • KEY_HANDLE	• KEY_HANDLE
密钥导入 98	导入一个密钥值	• 算法类型 • 权限 • 密钥值（明文或密文） • MAC 值 • TPK_ID • TVK_ID • KEY_HANDLE	• KEY_HANDLE
密钥导出 100	导出密钥值	• KEY_HANDLE • TPK_ID • TVK_ID	• 算法类型 • 权限 • 密钥值（明文或密文）

功能	说明	输入	输出
			MAC
从私钥计算公钥 104	从给定的私钥计算出公钥值并输出	- KEY_HANDLE - 参数	公钥值
密钥协商 96	非对称密钥协商	- 私钥 KEY_HANDLE - 公钥明文 - 协商参数 - 算法类型 - 权限 - KEY_HANDLE	KEY_HANDLE
密钥删除 102	删除指定密钥	KEY_HANDLE	-

说明:

1. TPK_ID, TVK_ID 分别代表密钥导入/导出时的加密密钥和校验密钥, 它们必须有 KEY_USAGE_TRANSPORT 权限;
2. 导入/导出时如果指定 TPK_ID=0xFFFFFFFF, 表示明文导入/导出, 相应的密钥必须具备明文导入/导出的权限;
3. 密钥生成/派生/导入/协商时, 输入的 KEY_HANDLE 的取值为:
 - 一个不存在的合法通用 KEY_HANDLE, 则会创建对应密钥, 并设置其 KEY_HANDLE 为输入的 KEY_HANDLE;
 - 一个已存在的合法通用 KEY_HANDLE, 则会覆盖对应密钥, 如果对应密钥已有写保护权限, 则覆盖会失败;
 - 0xFFFFFFFF, 表示自动分配一个有效的 KEY_HANDLE。
4. 当密钥操作会覆盖已存在密钥时, 会先检查被覆盖密钥的权限, 如果有冲突则操作失败。例如: 导入一个指定 KEY_HANDLE 的新密钥, 具有更多权限, 但旧密钥没有明文和密文可导入权限, 则导入失败
5. 当密钥操作会创建一个新密钥 (没有旧密钥可覆盖) 时, 不会检查新密钥指定的权限 (除非权限本身有冲突)。例如: 导入一个新密钥, 此密钥没有明文和密文可导入权限, 导入也能成功 (检查只针对被覆盖的密钥权限)。

6.4 OTP 密钥

OTP 密钥是加密存储在 OTP 中的密钥数据, 由硬件上电时解密到 KMU 中使用, 具体细节请参考 [1] 的相关章节。在 Mailbox 命令中可对 OTP 密钥进行引用¹², 其 KEY_HANDLE 为 0x0020000x, 其中 x 为 OTP 密钥的逻辑 ID (见下节), 例如 “eHSM Debug Key” 的 KEY_HANDLE 为 0x00200004。

¹² 此密钥必须配置用于通用密钥管理的权限值

6.4.1 密钥映射

OTP 中每个密钥占用 40 字节，在 [1] 中定义了多个存储在 OTP 中的密钥，考虑到减少 OTP 空间占用的需求，eHSM Firmware 支持密钥映射功能，即支持将多个功能映射到同一个 OTP 密钥值，例如 表 15：

表 15 OTP 密钥重映射示例

逻辑 ID	KEY_HANDLE	名称	物理 ID	说明
0	0x00200001	Chip Root Key	0	
1	0x00200002	Device Root Key	1	
2	0x00200003	User Root Key	2	
3	0x00200004	eHSM Debug Key	3	
4	0x00200005	eHSM FW Verify Key	3	
5	0x00200006	eHSM Encrypt Key	4	
6	0x00200007	eHSM Upgrade Encrypt Key	5	
7	0x00200008	eHSM Upgrade Verify Key	3	
8	0x00200009	eHSM Private Key	-	eHSM 内部暂未使用
9	0x0020000A	SOC Debug Key	6	
10	0x0020000B	SOC FW Verify Key	6	
11	0x0020000C	SOC Encrypt Key	7	
12	0x0020000D	SOC Upgrade Encrypt Key	8	
13	0x0020000E	SOC Upgrade Verify Key	6	
14	0x0020000F	SOC Private Key	-	eHSM 内部暂未使用
15	0x00200010	Secret Key	9	用于 RAM 密钥导入导出的加解密密钥
16	0x00200011	User Auth Key	9	

上例中，将 eHSM 的三个密钥合并成一个物理密钥，将 SOC 的三个密钥合并成一个物理密钥，同时两个 Private Key 没有映射（没有使用），因此，将 OTP 密钥减少到了 10 个¹³。

在 eHSM Firmware 源代码中可以定制上述映射表，以达到 OTP 密钥重映射的要求。对 OTP 密钥的配置通常包括：

1. 此 OTP 密钥是否可用于通用密钥管理（配置了对应的权限值，则可被 Mailbox 命令引用），如果禁止，则仅作为特殊用途（内部功能，如安全启动、升级、调试鉴权等）；
2. 配置此 OTP 密钥的权限属性时应当符合相应的约束条件，且强烈建议不支持导出权限。

6.4.2 OTP 密钥烧录（安装）

Firmware 支持 2 种密钥烧录命令：

1. **烧录随机密钥 69**：eHSM 会在内部生成一个随机密钥，使用对应的 Root Key 加密并烧录到 OTP 中。

¹³物理 ID 即密钥在 OTP 中的索引

2. [烧录指定密钥 71](#): 外部输入使用 RTL KEK 加密的密钥值, eHSM 会在内部使用 RTL KEK 解密, 然后重新使用 Root Key 加密并烧录到 OTP 中, 具体数据格式请查看命令说明。

OTP 密钥的烧录功能受生命周期控制, 请参考 [表 9](#)。

7 鉴权

一些功能需要鉴权后才可以使用：

1. 开启 eHSM 调试功能
2. 开启 SOC 调试功能
3. USER 模式控制功能

各功能对应的密钥如下表所示¹⁴

表 16 鉴权对应密钥

鉴权类型	OTP 密钥
eHSM 调试鉴权	eHSM Debug Key
SOC 调试鉴权	SOC Debug Key
USER 模式控制鉴权	User Auth Key

鉴权需要用到以下命令：

- [获取挑战值命令 59](#)
- [鉴权命令 61](#)

基本的鉴权流程如下图所示：

¹⁴OTP 密钥请参考 [表 15](#)

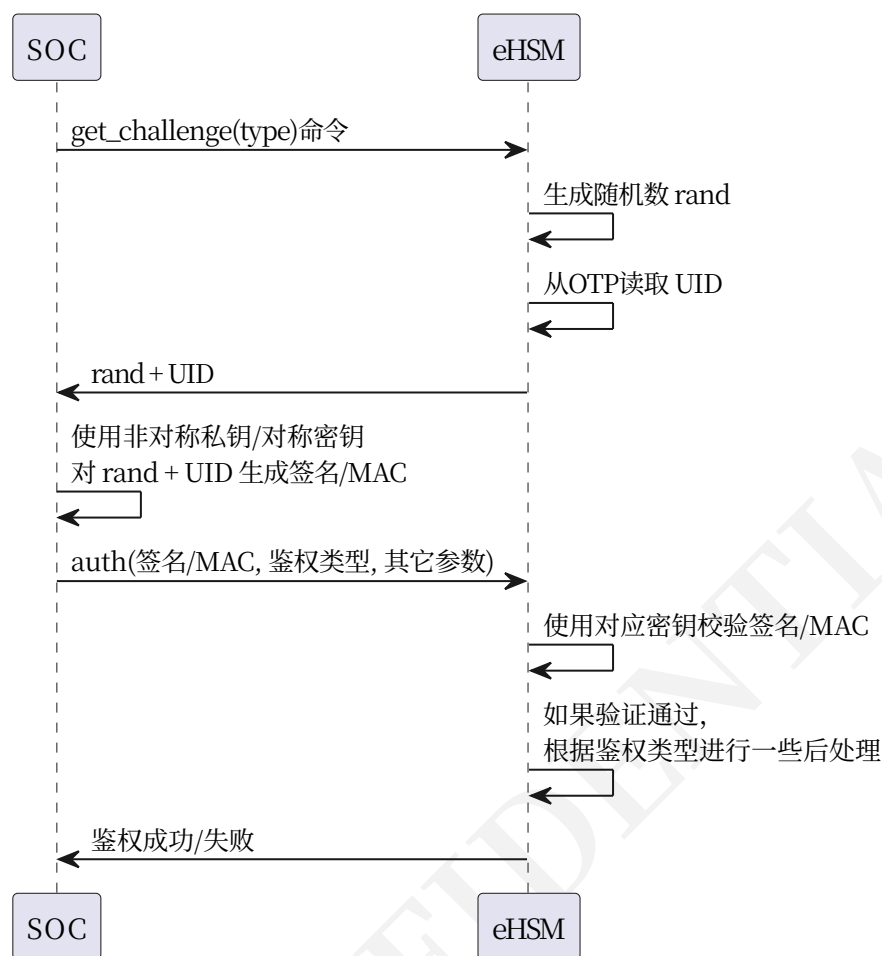


图 5 基本鉴权流程

7.1 调试鉴权

调试鉴权用于开启 eHSM 或 SOC 的调试能力，鉴权通过后，打开对应的调试端口：

- 对于 eHSM 调试，仅有一个端口
- 对于 SOC 调试，需要在调试鉴权命令中指定共计 129 个端口的使能位参数，端口对应的 bit 为 1 时打开调试，bit 为 0 的保持原状态

如果需要关闭调试端口，请使用 [关闭调试命令 63](#)，如果是 SOC 的调试端口，需在命令中指定共计 129 个端口的使能位参数，对应 bit 为 1 的端口被关闭，bit 为 0 的端口保持原状态。

此外，eHSM Firmware 还支持基于 ASCII 字符的指定协议进行调试鉴权，其鉴权流程与基于 Mailbox 的鉴权流程基本一致，可使用 UART 或 Mailbox 寄存器进行通信，但协议和数据格式有所不同，请参考 [附录 C](#) 获取协议细节。

7.2 USER 模式控制功能鉴权

目前 USER 模式控制鉴权的仅用于 USER 模式下切换生命周期，在鉴权成功后发送 [切换生命周期 76](#) 命令即可。

8 安全启动

本章节描述了 eHSM 安全启动设计，包括 eHSM Firmware 提供的安全启动能力，和对 SOC 侧的流程建议。

注：与 BootLoader 不同，eHSM Firmware 不支持校验 eHSM FW Image，只支持校验 SOC FW Image。

8.1 安全启动交互流程

1. SOC 等待 eHSM Firmware 启动完成，并处于可响应命令状态（HSM_READY）
2. SOC 发送 [镜像验证命令 67](#) 验证 SOC FW Image 的有效性（包含解密）

8.2 部署说明

SOC FW 可以支持两种类型的代码部署方式：

1. 基于 RAM 的部署方式：FW 镜像被解封 SoC 的 RAM 中执行
2. 基于 NVM 的部署方式：FW 镜像存储 NVM 中，并被 eHSM 校验后直接由 SoC 执行

基于 RAM 的部署：

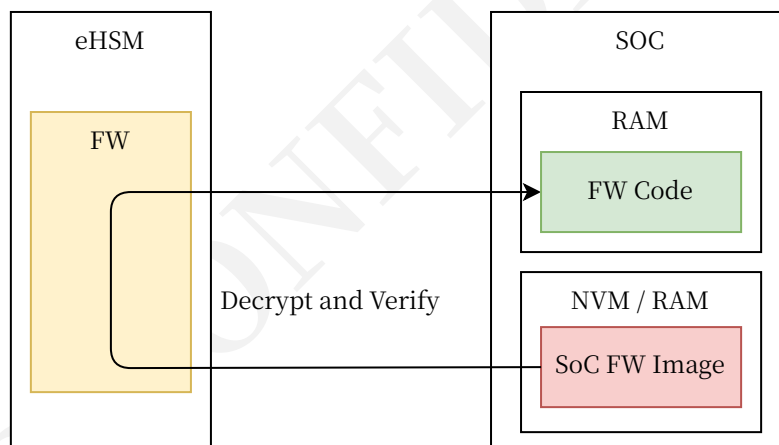


图 6 SoC FW 基于 RAM 的部署方式

在 RAM 部署方式下，由 SoC 将 Image 数据准备在存储器中，再通知 eHSM 进行验证。eHSM 将镜像数据读取并解密到 SoC 的 RAM 中，并校验数据的签名，最后回复 SoC 校验的结果。

在这种部署模式下，eHSM 支持将 SoC FW 镜像解密到原位置并校验，这要求镜像必须存储在 RAM 中。

基于 NVM 的部署：

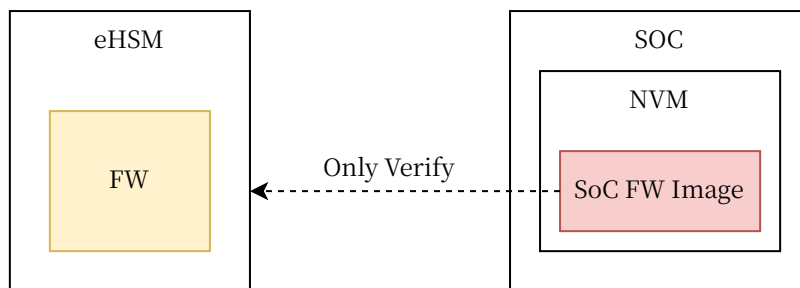


图 7 SoC FW 基于 NVM 的部署方式

在 NVM 部署方式下, 由 SOC 将 Image 数据准备在 NVM 存储器中, 再通知 eHSM 进行验证。eHSM 将镜像数据读取并直接校验数据的签名, 最后回复 SoC 校验的结果。

这种部署模式需要满足以下条件:

- eHSM CPU 可从 SOC 传入的地址从 NVM 中读取镜像数据
- 镜像只包含签名值, 不能加密

8.3 镜像格式

安全启动 Image 遵从以下格式:

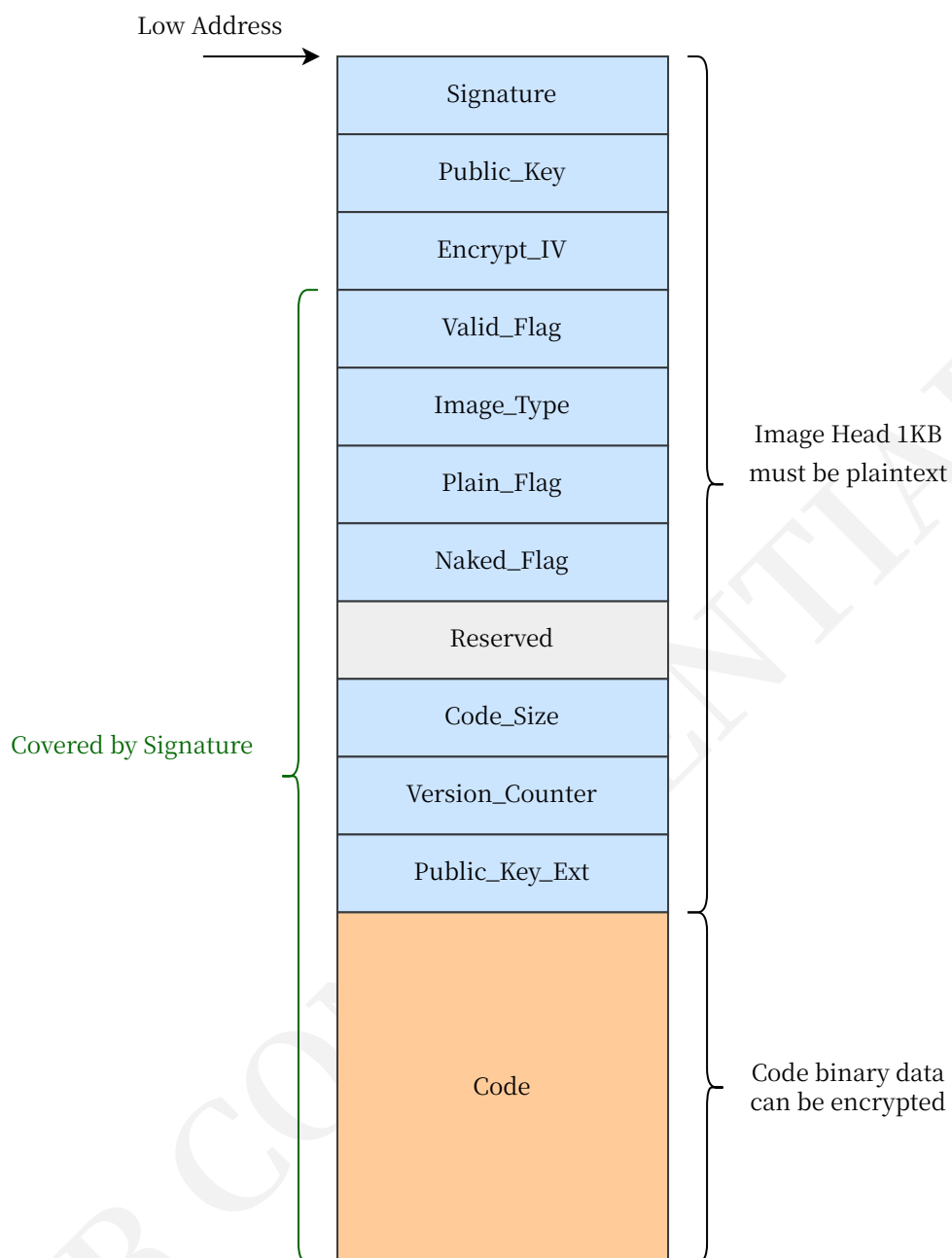


图 8 安全启动镜像格式示意图

表 17 安全启动镜像字段说明

字段	偏移(字节)	大小(字节)	说明
Signature	0	256	有效的数据长度由 OTP 中的 “EhsmCodeVerifyAlg” 或 “SocBootAlg” 定义的算法决定： • AES128/SM4-CMAC: 长度为 16 字节 • RSA2048: 长度为 256 字节 • RSA3072: 签名值的前 256 字节 • SM2: 长度为 64 字节 • ECC-P256R1: 长度为 64 字节
Public_Key	256	320	Public_Key 用于非对称算法。有效的长度由具体算法决定： • AES128/SM4 CMAC: 此字段无效 • RSA2048: 64 字节的 RSA(e) + 256 字节的 RSA(n) • RSA3072: 签名值的后 128 字节 + RSA(e) 64 字节 • SM2: 长度为 65 字节, 第一个字节 “0x04” 表示公钥值未压缩 • ECC-P256R1: 长度为 64 字节
Encrypt_IV	576	16	AES/SM4 CMAC 算法的 IV 值
Valid_Flag	592	4	代码是否有效标志： • 0x8E97645D: 代码有效 • 其它值: 代码无效
Image_Type	596	1	镜像类型： • 1: 使用 SOC 密钥加密/签名的 SOC 安全启动镜像 • 2: 使用 eHSM 密钥加密/签名的 SOC 安全启动镜像
Plain_Flag	597	1	代码部分是否明文： • 0: 代码部分是密文 • 1: 代码部分是明文
Naked_Flag	598	1	是否需要验证签名： • 0: 非裸镜像数据 • 1: 裸镜像数据, 除了此字段、Image_Type、Valid_Flag 字段和 Code 字段, 其它字段值都为全 0 注意: 仅在 TEST_MODE 和 DEVELOP_MODE 生命周期状态下支持裸镜像, 主要用于简化开发和验证; 其它状态下会报错。
Reserved	599	5	保留 (应当设置为全 0)
Code_Size	604	4	代码区域的大小
Version_Counter	608	16	128 位的单向版本计数
Public_Key_Ext	624	400	当验签密钥为 RSA3072 时, 此字段包含 384 字节的 RSA(n) 值, 后跟全 0; 其它验签密钥时此字段全部为 0
Code	1024	Code_Size	代码区域, 明文或密文

8.4 验证命令定义

请参考 [镜像验证命令 67](#)。

8.5 算法配置说明

支持以下算法：

- 验证算法：参考 [表 4](#) 中的 “SocBootAlg” 字段
- 加密算法：AES128-CBC 或 SM4-CBC（根据验证算法使用国际还是国密，决定使用 AES128 还是 SM4）

验证算法的类型在 OTP 中配置，参考 [表 4](#) 中的 “SocBootAlg” 字段。

验证算法使用的密钥在 OTP 中存储，SOC FW Image 验证时使用的密钥为 “SOC FW Verify Key” 和 “SOC Encrypt Key”，请参考 [\[1\]](#) 里的相关章节。

9 安全固件升级

本章节描述了 eHSM 安全固件升级功能的设计。

9.1 升级镜像制作

升级镜像是相对于 eHSM/SOC 的安全启动镜像而言，即在 eHSM/SOC 的安全启动镜像上附加了一层加密和签名。

升级镜像的制作分三步：

1. 生成明文安全启动镜像，使用 eHSM/SOC Verify Key 进行签名
2. 将明文安全启动镜像整个使用 eHSM/SOC Upgrade Encrypt Key 进行加密
3. 增加升级镜像头，并使用 eHSM/SOC Upgrade Verify Key 进行签名

参考下图：

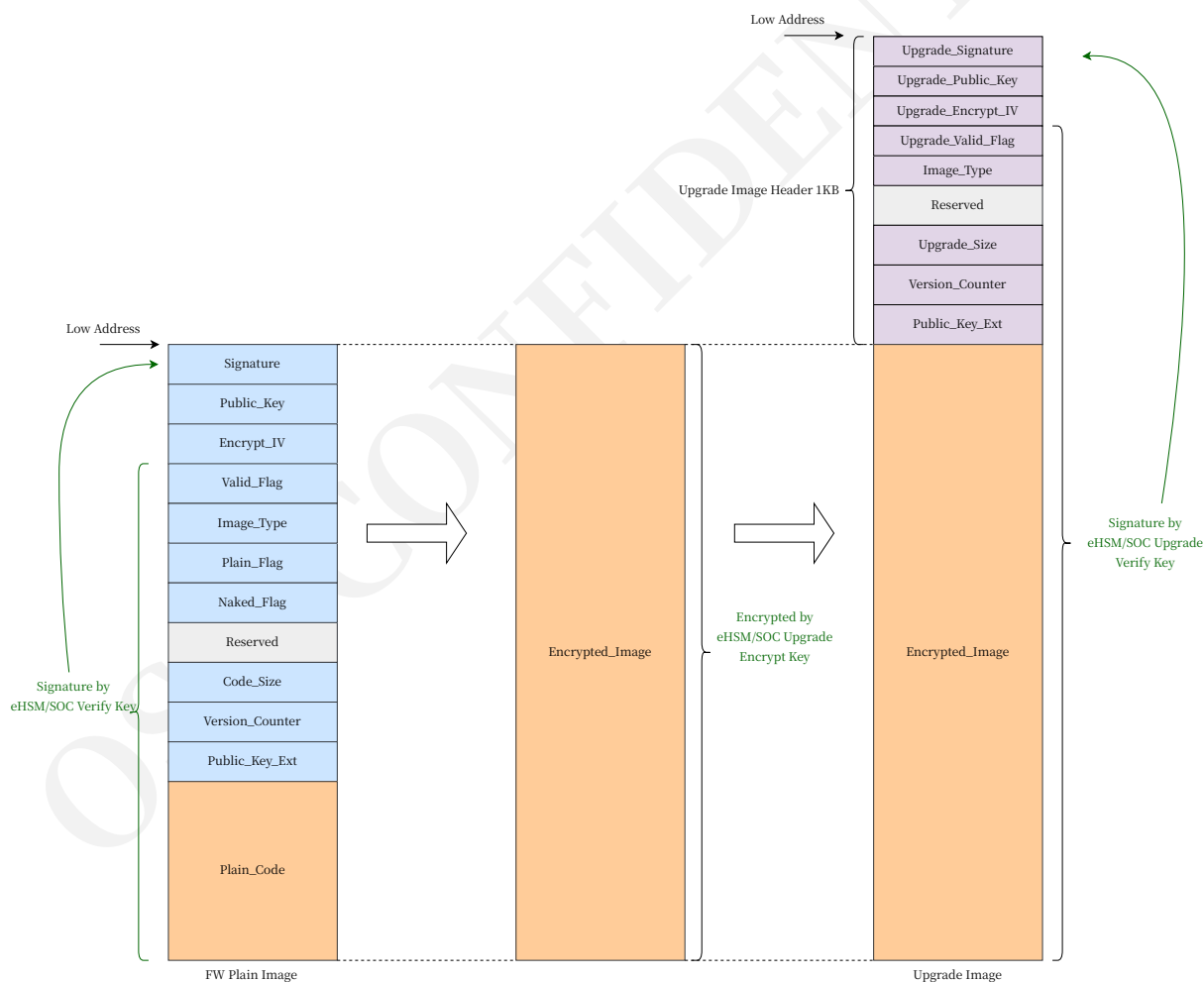


图 9 制作升级镜像

表 18 升级镜像字段说明

字段	偏移(字节)	大小(字节)	说明
Upgrade_Signature	0	256	有效的数据长度由 OTP 中的“EhsmCodeUpgradeAlg”或“SocUpgradeAlg”定义的算法决定： • AES128/SM4 CMAC: 长度为 16 字节 • RSA2048: 长度为 256 字节 • RSA3072: 签名值的前 256 字节 • SM2: 长度为 64 字节 • ECC-P256R1: 长度为 64 字节
Upgrade_Public_Key	256	320	Public_Key 用于非对称算法。有效的长度由具体算法决定： • AES128/SM4 CMAC: 此字段无效 • RSA2048: 64 字节的 RSA(e) + 256 字节的 RSA(n) • RSA3072: 签名值的后 128 字节 + RSA(e) 64 字节 • SM2: 长度为 65 字节, 第一个字节 “0x04” 表示公钥值未压缩 • ECC-P256R1: 长度为 64 字节
Upgrade_Encrypt_IV	576	16	AES/SM4 CMAC 算法的 IV 值
Upgrade_Valid_Flag	592	4	代码是否有效标志： • 0x71689BA2: 镜像有效 • 其它值: 镜像无效
Image_Type	596	1	镜像类型： • 0: eHSM 升级镜像（必须使用 eHSM Upgrade 密钥加密/签名） • 1: SOC 升级镜像, 使用 SOC Upgrade 密钥加密/签名 • 2: SOC 升级镜像, 使用 eHSM Upgrade 密钥加密/签名
Reserved	597	7	保留 (应当设置为全 0)
Upgrade_Size	604	4	加密镜像区域的大小
Version_Counter	608	16	128 位的单向版本计数
Public_Key_Ext	624	400	当验签密钥为 RSA3072 时, 此字段包含 384 字节的 RSA(n) 值, 后跟全 0; 其它验签密钥时此字段全部为 0
Encrypted_Image	1024	Upgrade_Size	加密的代码镜像

9.2 升级镜像安装

安装升级镜像的命令请参考[固件升级命令 65](#)。

eHSM 升级流程与制作流程相反：

1. 根据升级镜像中的 “Image_Type” 字段，使用 “eHSM Upgrade Verify Key” 或 “SOC Upgrade Verify Key” 验证升级镜像
2. 根据升级镜像中的 “Image_Type” 字段，使用 “eHSM Upgrade Encrypt Key” 或 “SOC Upgrade Encrypt Key” 解密镜像的 “Encrypt_Image” 字段，得到固件明文镜像
3. 最后，根据固件明文镜像中的 “Plain_Flag” 字段，决定是否要加密固件镜像的 “Plain_Code” 字段；如果需要加密，根据固件明文镜像中的 “Image_Type” 字段，决定使用 “eHSM Encrypt Key” 还是 “SOC Encrypt Key” 进行加密

升级完成后的固件镜像是安全启动镜像，SOC 端应当使用 eHSM 的镜像验证命令对安全启动镜像进行验证，成功后由 SOC 端负责存储，并在后续启动时使用中提供给 eHSM 进行验证。

9.3 版本管理

升级镜像包含 128 位的单向版本计数器，用于防止降级攻击。每次升级时，新镜像的版本计数器必须大于等于当前安装镜像的版本计数器。

9.4 配置说明

9.4.1 升级算法配置

支持以下算法：

- 验证算法：参考 表 3 中的 “EhsmCodeUpgradeAlg” 字段以及表 表 4 中的 “SocUpgradeAlg” 字段
- 加密算法：AES128-CBC/SM4-CBC（根据验证算法使用国际还是国密，决定使用 AES128 还是 SM4）

验证算法使用的密钥在 OTP 中存储，eHSM FW 使用的密钥为 “eHSM FW Upgrade Verify Key” 和 “eHSM Upgrade Encrypt Key”，SOC FW 使用的密钥为 “SOC FW Upgrade Verify Key” 和 “SOC Upgrade Encrypt Key”，请参考 表 7。

10 Mailbox 通信协议

10.1 硬件说明

eHSM 软件通过硬件的 Mailbox 通道交互数据信息，在硬件设计中，

- 共支持最多 16 个 mailbox 通道
- 其中下行方向（从 SOC 到 eHSM）每个 Mailbox 通道有 2 个数据寄存器 `MB_S2H_INFO[0]`、`MB_S2H_INFO[1]`，1 个状态寄存器 `MB_S2H_NOTE`
- 上行方向（从 eHSM 到 SOC）每个 Mailbox 通道有 2 个数据寄存器 `MB_H2S_INFO[0]`、`MB_H2S_INFO[1]`，1 个状态寄存器 `MB_H2S_NOTE`。

依靠硬件提供的寄存器，以及 eHSM 可以访问 SOC RAM 的特性¹⁵，eHSM 遵循相应的协议来响应上位机的命令。¹⁶

10.2 协议说明

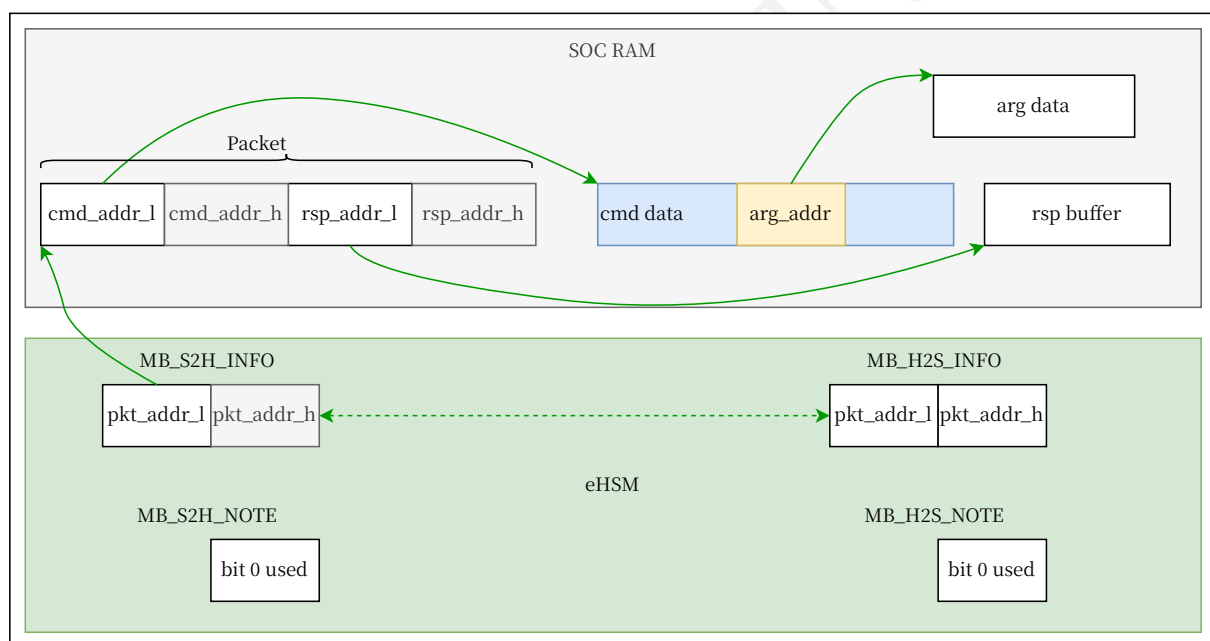


图 10 MailBox 协议数据流

说明:

- Mailbox `MB_S2H_INFO` 寄存器中存放命令 packet 的地址
- `MB_S2H_INFO` 中的 packet 地址指向 SOC RAM 中的一个 packet 结构体，结构体包含命令数据的起始地址和响应数据 buffer 的起始地址
- 设置 `MB_S2H_NOTE` 寄存器的 bit0 为 1，用于通知 eHSM 接收数据
- `MB_H2S_NOTE` 的 bit0 变为 1 时，表示命令处理完成，`MB_H2S_INFO` 中保存了 packet 地址，表明处理完成的是哪个 packet

¹⁵所有命令数据、响应 buffer 都必须存放在 eHSM 可访问的 SOC RAM 中，Mailbox 仅用于发送地址。

¹⁶硬件的 Mailbox 设计请参考 [1]。

- cmd data 按实际命令大小准备，设计 Mailbox 命令时不会超过 128 字节
- rsp buffer 大小为 20 字节，eHSM 返回的数据不会超过此大小

注：

- 图中 RAM 数据和寄存器都是左边为低地址，右边为高地址
- 如果 SOC RAM 是 32 位地址，则对应的 addr_h 填 0
- 图中的数据都是小端字节序

协议的基本流程：

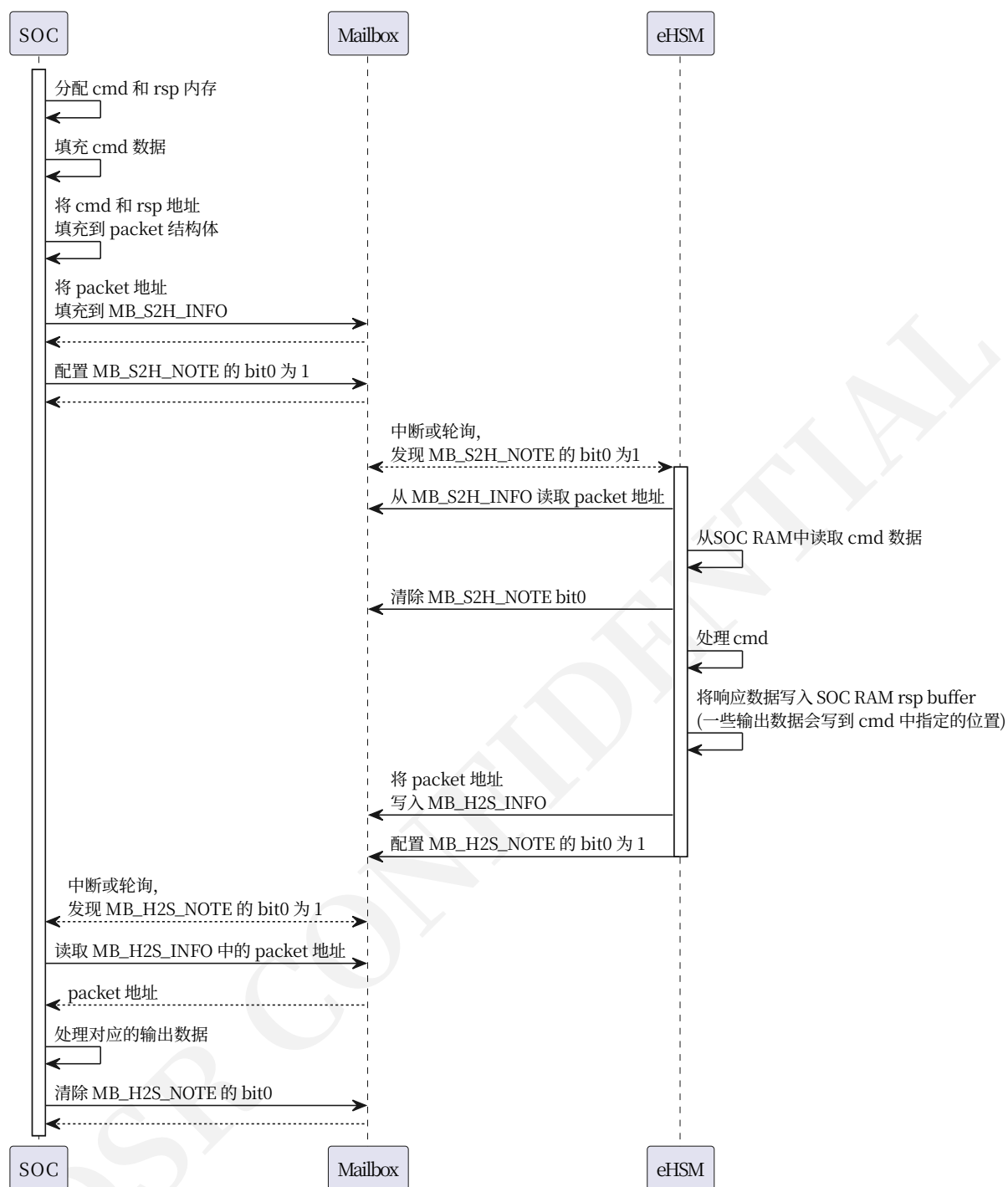


图 11 Mailbox 协议流程

A Mailbox 命令

所有命令、结构体中的定义的超 1 字节宽的数据，均为小端存储，另有说明的除外。

A.1 全局定义

算法的处理模式。

表 19 枚举 process_mode 定义

枚举名称	值	说明
START	1	三段式处理的开始阶段
UPDATE	2	三段式处理的更新数据阶段
STREAMSTART	3	三段式处理的开始和更新阶段
FINISH	4	三段式处理的结束阶段
ONE-PASS	7	一次性处理

MAC 生成/验证模式。

表 20 枚举 mac_direction 定义

枚举名称	值	说明
MAC-GEN	0	生成 MAC 值
MAC-VRY	1	校验 MAC 值

指令的密钥使用方式。

表 21 枚举 key_type 定义

枚举名称	值	说明
KEY-HANDLE	0	算法指令使用密钥句柄
PLAIN-KEY	1	算法指令使用明文密钥

签名验证/生成模式。

表 22 枚举 sig_direction 定义

枚举名称	值	说明
SIG-VRY	0	校验签名值
SIG-GEN	1	生成签名值

加密/解密模式。

表 23 枚举 cipher_direction 定义

枚举名称	值	说明
CIPHER-ENC	0	加密
CIPHER-DEC	1	解密

消息/摘要。

表 24 枚举 msg_type 定义

枚举名称	值	说明
MSG-TYPE-MESSAGE	0	消息数据
MSG-TYPE-DIGEST	1	摘要数据

鉴权挑战类型。

表 25 枚举 auth_challenge_type 定义

枚举名称	值	说明
EHSM_CHALLENGE_TYPE_EHSM_DEBUG	1	eHSM 调试鉴权
EHSM_CHALLENGE_TYPE_SOC_DEBUG	3	SOC 调试鉴权
EHSM_CHALLENGE_TYPE_USER_AUTH	4	USER 模式身份验证

密钥类型 ID。

表 26 枚举 key_algo_id 定义

枚举名称	值	说明
DES	1	DES 密钥
TDES_128	2	TDES-128 密钥
TDES_192	3	TDES-192 密钥
AES_128	4	AES-128 密钥
AES_192	5	AES-192 密钥
AES_256	6	AES-256 密钥
SM4	7	SM4 密钥
SM2	8	SM2 密钥
RSA_1024	9	RSA1024 密钥
RSA_2048	10	RSA2048 密钥
RSA_3072	11	RSA3072 密钥
RSA_4096	12	RSA4096 密钥
RSA_1024_CRT	13	RSA1024-CRT 密钥
RSA_2048_CRT	14	RSA2048-CRT 密钥
RSA_3072_CRT	15	RSA3072-CRT 密钥
RSA_4096_CRT	16	RSA4096-CRT 密钥
DH	17	DH 密钥
ECC_BRAINPOOLP_160R1	18	ECC Brainpool p160r1 曲线密钥

枚举名称	值	说明
ECC_BRAINPOOLP_192R1	19	ECC Brainpool p192r1 曲线密钥
ECC_BRAINPOOLP_224R1	20	ECC Brainpool p224r1 曲线密钥
ECC_BRAINPOOLP_256R1	21	ECC Brainpool p256r1 曲线密钥
ECC_BRAINPOOLP_320R1	22	ECC Brainpool p320r1 曲线密钥
ECC_BRAINPOOLP_384R1	23	ECC Brainpool p384r1 曲线密钥
ECC_BRAINPOOLP_512R1	24	ECC Brainpool p512r1 曲线密钥
ECC_SECP_192R1	25	ECC secp192r1 曲线密钥
ECC_SECP_224R1	26	ECC secp224r1 曲线密钥
ECC_SECP_256R1	27	ECC secp256r1 曲线密钥
ECC_SECP_384R1	28	ECC secp384r1 曲线密钥
ECC_SECP_521R1	29	ECC secp521r1 曲线密钥
ED25519	30	爱德华曲线密钥
X25519	31	蒙哥马利曲线密钥
SM4_XTS	32	SM4-XTS 密钥
AES_128_XTS	33	AES-128-XTS 密钥
AES_192_XTS	34	AES-192-XTS 密钥
AES_256_XTS	35	AES-256-XTS 密钥
HMAC	37	HMAC 密钥
ECC_SECP_160K1	38	ECC secp160k1 密钥
ECC_SECP_192K1	39	ECC secp192k1 密钥
ECC_SECP_224K1	40	ECC secp224k1 密钥
ECC_SECP_256K1	41	ECC secp256k1 密钥
ECC_SECP_160R1	46	ECC secp160r1 曲线密钥
ECC_SECP_160R2	47	ECC secp160r2 曲线密钥

密钥权限。

表 27 枚举 usage_permit_flags 定义

枚举名称	值	说明
KEY_USAGE_NONE	0	
KEY_USAGE_SIGN	1	密钥可用于签名或生成 MAC/HMAC
KEY_USAGE_VERIFY	2	密钥可用于验签或验证 MAC/HMAC
KEY_USAGE_ENCRYPT	4	密钥可用于加密
KEY_USAGE_DECRYPT	8	密钥可用于解密
KEY_USAGE_KEYCREATION	128	密钥可用于派生或协商新密钥
KEY_USAGE_CREATION_TRANSP_KEY	256	密钥可用于派生传输密钥
KEY_USAGE_TRANSPORT	1024	密钥可作为传输密钥使用
KEY_PERMIT_REMOVE	2048	密钥可以被移除
KEY_PERMIT_IMPORT_PLAINTEXT	4096	密钥可以被明文导入
KEY_PERMIT_IMPORT_CIPHERTEXT	8192	密钥可以被密文导入

枚举名称	值	说明
KEY_PERMIT_EXPORT_PLAINTEXT	16384	密钥可以被明文导出
KEY_PERMIT_EXPORT_CIPHERTEXT	32768	密钥可以被密文导出
KEY_PERMIT_WR_PRT	524288	密钥写保护（不能覆盖）

A.2 对称算法

这些命令定义了对称的加解密算法，包括普通的加解密和 AEAD、流加密等算法。

A.2.1 symm_cipher

对称加解密算法操作。

表 28 结构体 CMD-SYMM-CIPHER 定义

位置	字段名称	说明
0-1	cmd_id	0x0101
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	加密方向。参考 表 23。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-14	padding	填充类型。选项： • 0x00: 无填充 • 0x02: PKCS7 填充 • 0x03: ONEWITHZEROS (ISO-7816-4) START/UPDATE/FINISH/ONE-PASS 模式应设置。
15-15	algorithm	对称加密算法。选项： • 0x00: DES 对称加密算法 • 0x01: TDES-128 对称加密算法 • 0x02: TDES-192 对称加密算法 • 0x05: AES-128 对称加密算法 • 0x06: AES-192 对称加密算法 • 0x07: AES-256 对称加密算法 • 0x08: 国密 SM4 对称加密算法 • 其他值为保留值 START/UPDATE/FINISH/ONE-PASS 模式应设置。
16-16	cipher_mode	对称加密模式。选项： • 0x01: 对称加密的 ECB 模式 • 0x02: 对称加密的 XTS 模式 • 0x03: 对称加密的 CBC 模式 • 0x04: 对称加密的 CFB 模式

位置	字段名称	说明
		• 0x05: 对称加密的 OFB 模式 • 0x06: 对称加密的 CTR 模式 • 其他值为保留值 START/UPDATE/FINISH/ONE-PASS 模式应设置。
17-19	reserved1	-
20-23	key_handle	用于操作的密钥, 当 key_type 为 0 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved2	-
36-43	input_addr	输入明文(加密模式)或密文(解密模式)的地址。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
44-47	input_size	输入文本的大小。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
48-55	output_addr	输出明文或密文的地址。UPDATE/FINISH/ONE-PASS 模式应设置。
56-59	output_size	输出缓冲区的大小。UPDATE/FINISH/ONE-PASS 模式应设置。
60-67	iv_addr	初始化向量/Nonce 的地址。 • 对齐: 32 位。 • 在 ECB 模式下, iv_addr 被忽略。 • 在 CTR 模式下, IV 是初始计数器。 • 在 XTS 模式下, IV 是初始 i 值。 START/ONE-PASS 模式应设置。
68-71	iv_size	初始化向量/Nonce 的大小。START/ONE-PASS 模式应设置。
72-79	context	加密上下文。START/UPDATE/FINISH 模式应设置。
80-83	context_size	加密上下文的长度。START/UPDATE/FINISH 模式应设置。
84-91	plain_key_addr	明文密钥, key_type 为 1 时有效 START/UPDATE/FINISH/ONE-PASS 模式应设置。
92-95	plain_key_size	明文密钥长度, key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。

响应:

表 29 结构体 RSP-SYMM-CIPHER 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。
4-7	output_size	输出明文或密文的大小

A.2.2 aead_gcm

AEAD 的 GCM 模式加解密命令。

表 30 结构体 CMD-AEAD-GCM 定义

位置	字段名称	说明
0-1	cmd_id	0x0102
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	加密方向。参考 表 23。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-14	reserved1	-
15-15	algorithm	对称加密算法。选项： • 0x05: AES_128 • 0x06: AES_192 • 0x07: AES_256 • 0x08: SM4 • 其他值为保留值 START/UPDATE/FINISH/ONE-PASS 模式应设置。
16-19	reserved2	-
20-23	key_handle	用于操作的密钥，key_type 为 0 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved3	-
36-43	input_addr	GCM 输入数据的地址。input_addr 是 AEAD 输入的地址，可以是明文或密文，取决于方向。在 ONE-PASS 模式下和流模式中的 UPDATA 时设置。在 FINAL 模式中为可选。
44-47	input_size	输入明文或密文的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
48-55	output_addr	GCM 输出数据的地址。output_addr 是 AEAD 输出的地址，可以是明文或密文，取决于方向。在 ONE-PASS 模式下和流模式中的 UPDATA 时设置。在 FINAL 模式中为可选。
56-59	output_size	输出缓冲区的大小。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
60-67	aad_addr	GCM 关联数据的地址。aad_addr 是关联数据的地址，应在 ONE-PASS 模式下以及流模式中的 START 或 STREAMSTART 时设置为有效。如果 AEAD 请求命令不需要输入关联数据，请将其设置为 0。
68-71	aad_size	关联数据的字节长度，不应大于 128。START/ONE-PASS 模式应设置。
72-79	tag_addr	GCM 标签的地址。tag_addr 是标签的地址，在解密时为输入，在加密时为输出。在 ONE-PASS 模式和流模式中的 FINISH 模式中应设置。
80-83	tag_size	标签长度。标签大小应在 1 到 16 的范围内。FINISH/ONE-PASS 模式应设置。
84-91	iv_addr	初始化向量/Nonce 的地址。START/ONE-PASS 模式应设置。
92-95	iv_size	IV 的字节长度应为 12 字节。START/ONE-PASS 模式应设置。
96-103	context	加密上下文。START/UPDATE/FINISH 模式应设置。
104-107	context_size	加密上下文的长度。START/UPDATE/FINISH 模式应设置。
108-115	plain_key_addr	明文密钥，key_type 为 1 时有效 START/UPDATE/FINISH/ONE-PASS 模式应设置。
116-119	plain_key_size	明文密钥长度，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。

响应:

表 31 结构体 RSP-AEAD-GCM 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	output_size	输出明文或密文的大小
8-11	verification	解密服务中标签验证的结果。0 表示成功, 其他表示失败。加密时忽略。

A.2.3 aead_ccm

AEAD 的 CCM 模式加解密命令。

表 32 结构体 CMD-AEAD-CCM 定义

位置	字段名称	说明
0-1	cmd_id	0x0103
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	加密方向。参考 表 23。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-14	reserved1	-
15-15	algorithm	对称加密算法。选项: • 0x05: AES_128 • 0x06: AES_192 • 0x07: AES_256 • 0x08: SM4 • 其他值为保留值 START/UPDATE/FINISH/ONE-PASS 模式应设置。
16-19	reserved2	-
20-23	key_handle	用于操作的密钥, 当 key_type 为 0 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved3	-
36-43	input_addr	CCM 输入数据的地址。input_addr 是 AEAD 输入的地址, 可以是明文或密文, 取决于方向。在 ONE-PASS 模式下和流模式中的 UPDATA 时设置。在 FINAL 模式中为可选。
44-47	input_size	输入明文或密文的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
48-55	output_addr	CCM 输入结构数据的地址。output_addr 是 AEAD 输出的地址, 可以是明文或密文, 取决于方向。在 ONE-PASS 模式下和流模式中的 UPDATA 时设置。在 FINAL 模式中为可选。
56-59	output_size	输出缓冲区的大小。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
60-67	aad_addr	CCM 关联数据的地址。aad_addr 是关联数据的地址, 应在 ONE-PASS 模式下以及流模式中的 START 或 STREAMSTART 时设置为有效。如果 AEAD 请求命令不需要输入关联数据, 请将其设置为 0。

位置	字段名称	说明
68-71	aad_size	关联数据的字节长度，不应大于 128。START/ONE-PASS 模式应设置。
72-79	tag_addr	CCM 标签的地址。tag_addr 是标签的地址，在解密时为输入，在加密时为输出。在 ONE-PASS 模式和流模式中的 FINISH 模式中应设置。
80-83	tag_size	标签长度。标签大小应在 1 到 16 的范围内。FINISH/ONE-PASS 模式应设置。
84-91	nonce_addr	nonce 的地址。START/ONE-PASS 模式应设置。
92-95	nonce_size	nonce 的字节长度。范围在 [7, 13]。START/ONE-PASS 模式应设置。
96-103	context	加密上下文。START/UPDATE/FINISH 模式应设置。
104-107	context_size	加密上下文的长度。START/UPDATE/FINISH 模式应设置。
108-115	plain_key_addr	明文密钥，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
116-119	plain_key_size	明文密钥长度，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。

响应：

表 33 结构体 RSP-AEAD-CCM 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。
4-7	output_size	输出明文或密文的大小
8-11	verification	解密服务中标签验证的结果。0 表示成功，其他表示失败。加密时忽略。

A.3 数据摘要算法

包括 HASH、HMAC 算法命令

A.3.1 hash

HASH 算法命令。

表 34 结构体 CMD-HASH 定义

位置	字段名称	说明
0-1	cmd_id	0x0301
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	process_mode	处理模式。参考 表 19。
13-14	reserved1	-
15-15	algorithm	哈希算法类型。选项： • 0x00: SM3 • 0x01: MD5 • 0x02: SHA256 • 0x03: SHA384

位置	字段名称	说明
		• 0x04: SHA512 • 0x05: SHA1 • 0x06: SHA224 • 0x07: SHA512_224 • 0x08: SHA512_256 • 0x09: SHA3_224 • 0x0a: SHA3_256 • 0x0b: SHA3_384 • 0x0c: SHA3_512 START/UPDATE/FINISH/ONE-PASS 模式应设置。
16-35	reserved2	-
36-43	data	用于生成哈希值的消息数据的地址。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
44-47	data_size	消息数据的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
48-55	digest	用于存储 HASH 值的地址 FINISH/ONE-PASS 模式应设置。
56-59	digest_size	HASH 缓冲区的长度 FINISH/ONE-PASS 模式应设置。
60-71	reserved3	-
72-79	hash_ctx	HASH 上下文的地址。START/UPDATE/FINISH 模式应设置。
80-83	hash_ctx_size	上下文的长度。START/UPDATE/FINISH 模式应设置。

响应:

表 35 结构体 RSP-HASH 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。
4-7	digest_size	输出 HASH 值的大小

A.3.2 hmac

HMAC 算法命令。

表 36 结构体 CMD-HMAC 定义

位置	字段名称	说明
0-1	cmd_id	0x0302
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	HMAC 方向。参考 表 20。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-14	reserved1	-

位置	字段名称	说明
15-15	algorithm	用于 HMAC 的 HASH 算法类型。选项： • 0x00: SM3 • 0x01: MD5 • 0x02: SHA256 • 0x03: SHA384 • 0x04: SHA512 • 0x05: SHA1 • 0x06: SHA224 • 0x07: SHA512_224 • 0x08: SHA512_256 • 0x09: SHA3_224 • 0x0a: SHA3_256 • 0x0b: SHA3_384 • 0x0c: SHA3_512 START/UPDATE/FINISH/ONE-PASS 模式应设置。
16-19	reserved2	-
20-23	key_handle	用于生成 HMAC 的密钥，当 key_type 为 0 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved3	-
36-43	data	用于生成哈希值的消息数据的地址。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
44-47	data_size	消息数据的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
48-55	digest	用于 HMAC 值的地址。FINISH/ONE-PASS 模式应设置。
56-59	digest_size	HMAC 缓冲区的长度。FINISH/ONE-PASS 模式应设置。
60-71	reserved4	-
72-79	hmac_ctx	HMAC 上下文的地址。START/UPDATE/FINISH 模式应设置。
80-83	hmac_ctx_size	上下文的长度。START/UPDATE/FINISH 模式应设置。
84-91	plain_key_addr	明文密钥，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
92-95	plain_key_size	明文密钥长度，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。

响应：

表 37 结构体 RSP-HMAC 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	digest_size	输出 HMAC 值的大小
8-11	verification	验证结果。0 表示成功，其他表示失败。生成 HMAC 时忽略。

A.4 MAC 算法

SKE MAC 算法命令

A.4.1 mac

MAC 生成或验证命令。

表 38 结构体 CMD-MAC 定义

位置	字段名称	说明
0-1	cmd_id	0x0201
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	MAC 方向。参考 表 20。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-14	reserved1	-
15-15	algorithm	MAC 使用的对称算法。选项： • 0x00: DES • 0x01: TDES_128 • 0x02: TDES_192 • 0x05: AES_128 • 0x06: AES_192 • 0x07: AES_256 • 0x08: SM4 • 其他值为保留值 START/UPDATE/FINISH/ONE-PASS 模式应设置。
16-16	cipher_mode	MAC 加密模式。选项： • 0x07: CMAC • 0x08: CBC_MAC • 0x09: GMAC START/UPDATE/FINISH/ONE-PASS 模式应设置。
17-19	reserved2	-
20-23	key_handle	用于生成/验证 MAC 的密钥，当 key_type 为 0 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved3	-
36-43	data	用于生成/验证 MAC 的消息数据的地址。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
44-47	data_size	消息数据的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
48-55	mac	生成或验证后的 MAC 值的地址。FINISH/ONE-PASS 模式应设置。
56-59	mac_size	MAC 缓冲区的长度或生成的 MAC 值的长度。FINISH/ONE-PASS 模式应设置。

位置	字段名称	说明
60-67	iv_addr	GMAC 的 IV 地址。仅支持 GMAC，且大小应为 12 字节。START/ONE-PASS 模式应设置。
68-71	reserved4	-
72-79	mac_ctx	MAC 的上下文地址。START/UPDATE/FINISH 模式应设置。
80-83	mac_ctx_size	上下文的长度。START/UPDATE/FINISH 模式应设置。
84-91	plain_key_addr	明文密钥，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
92-95	plain_key_size	明文密钥长度，key_type 为 1 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。

响应：

表 39 结构体 RSP-MAC 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。
4-7	mac_size	输出 MAC 的大小。
8-11	verification	验证结果。0 表示成功，其他表示失败。生成 MAC 时忽略。

A.5 非对称算法

非对称算法命令，包括 SM2、RSA、ECDSA、SM9 等非对称的签名及加解密算法。

SM2 明文密钥数据结构

表 40 结构体 sm2_key 定义

位置	字段名称	说明
0-31	privkey	SM2 私钥。
32-96	pubkey	SM2 公钥。

A.5.1 sm2_cipher

SM2 加解密算法命令。

表 41 结构体 CMD-SM2-CIPHER 定义

位置	字段名称	说明
0-1	cmd_id	0x0401
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	reserved1	-
13-13	direction	加密方向。参考 表 23。
14-19	reserved2	-

位置	字段名称	说明
20-23	key_handle	用于操作的密钥，当 key_type 为 0 时有效。
24-35	reserved3	-
36-43	input_addr	输入数据的地址。 - 加密时存储明文。 - 解密时存储密文。 - 密文分为三部分。C1 是曲线上的一点，65 字节（第一个字节固定为 0x04，表示非压缩点，即包含完整的 x 和 y 坐标）；C2 与明文长度相同；C3 是检查值，用于确保完整性，采用 SM3 算法，32 字节。 - 密文顺序：C1C3C2 - 解密后明文的字节长度，通常为密文长度 -97。
44-47	input_size	输入数据的长度。选项： - 加密时存储明文的长度。应当大于 0 且不超过 927。 - 解密时存储密文的长度。应当大于 97 且不超过 1024。
48-55	output_addr	输出数据的地址。 - 加密时存储明文。 - 解密时存储密文。 - 密文分为三部分。C1 是曲线上的一点，65 字节（第一个字节固定为 0x04，表示非压缩点，即包含完整的 x 和 y 坐标）；C2 与明文长度相同；C3 是检查值，用于确保完整性，采用 SM3 算法，32 字节。 - 密文顺序：C1C3C2 - 解密后明文的字节长度，通常为密文长度 -97。
56-59	output_size	输出缓冲区的长度。
60-67	plain_key_addr	明文密钥，key_type 为 1 时有效。加密只需要传公钥，私钥会被忽略。解密只需要传私钥，公钥会被忽略。参考 表 40。

响应：

表 42 结构体 RSP-SM2-CIPHER 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。
4-7	output_size	输出明文或密文的大小。

A.5.2 sm2_sign

SM2 签名/验证命令。

表 43 结构体 CMD-SM2-SIGN 定义

位置	字段名称	说明
0-1	cmd_id	0x0402

位置	字段名称	说明
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	签名方向。参考 表 22。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-18	reserved1	-
19-19	msg_type	数据类型，表示 msg_addr 地址指向的数据是原始消息还是消息摘要(SM2 E 值),仅 onepass 模式支持消息摘要参考 表 24。
20-23	key_handle	用于操作的密钥。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved2	-
36-43	msg_addr	输入消息的地址。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
44-47	msg_size	输入消息的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。如果 msg_type 值为 0x01,msg_size 必须是 32 字节
48-55	sign_addr	签名值的地址。FINISH/ONE-PASS 模式应设置。
56-59	sign_size	签名缓冲区或签名值的大小。FINISH/ONE-PASS 模式应设置。
60-71	reserved3	-
72-79	sign_ctx	签名或验证的上下文地址。START/UPDATE/FINISH 模式应设置。
80-83	sign_ctx_size	上下文的长度。START/UPDATE/FINISH 模式应设置。
84-91	plain_key_addr	明文密钥，key_type 为 1 时有效。签名只需要传私钥，公钥会被忽略，验签只需要传公钥，私钥会被忽略，START/UPDATE/FINISH/ONE-PASS 模式应设置。参考 表 40。

响应：

表 44 结构体 RSP-SM2-SIGN 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。
4-7	sign_size	输出签名的大小。验证服务中忽略。
8-11	verification	验证结果。0 表示成功，其他表示失败。签名服务中忽略。

RSA 明文密钥数据结构

表 45 结构体 rsa_key 定义

位置	字段名称	说明
0-0	crt_mode	CRT 模式，1 代表私钥时 CRT 模式。
1-3	reserved0	-
4-7	n_byte_sz	n 值的字节数，必须是 4 的倍数。
8-11	e_byte_sz	e 值的字节数，必须是 4 的倍数。
12-19	n	n 值的存储地址
20-27	e	e 值的存储地址
28-35	d	d 值的存储地址

位置	字段名称	说明
36-43	p	p 值的存储地址
44-51	q	q 值的存储地址
52-59	dp	dp 值的存储地址
60-67	dq	dq 值的存储地址
68-75	u	u 值的存储地址

A.5.3 Rsa_cipher

RSA 加密/解密命令。

表 46 结构体 CMD-RSA-CIPHER 定义

位置	字段名称	说明
0-1	cmd_id	0x0501
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。
12-12	reserved1	-
13-13	direction	加密方向。参考 表 23。
14-19	reserved2	-
20-23	key_handle	用于操作的密钥，当 key_type 为 0 时有效。
24-35	reserved3	-
36-43	input_addr	输入数据的地址。
44-47	input_size	输入数据的长度。
48-55	output_addr	输出数据的地址。
56-59	output_size	输出缓冲区的长度。
60-67	plain_key_addr	明文密钥，key_type 为 1 时有效。解密需要传公钥和私钥，加密只需要传公钥，私钥字段被忽略。参考 表 45。

响应：

表 47 结构体 RSP-RSA-CIPHER 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。
4-7	output_size	输出明文或密文的大小。

A.5.4 Rsa_sign

RSA 签名/验证命令。

表 48 结构体 CMD-RSA-SIGN 定义

位置	字段名称	说明
0-1	cmd_id	0x0502
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	签名方向。参考 表 22。START/UPDATE/FINISH/ONE-PASS 模式应设置。
14-14	rsa_padding_type	RSA 填充类型。选项： • 0x00: 不填充 • 0x01: RSASSA PSS 填充 START/UPDATE/FINISH/ONE-PASS 模式应设置。
15-15	algorithm	用于签名或验证的哈希算法。选项： • 0x02: SHA256 • 0x03: SHA384 • 0x04: SHA512 • 0x05: SHA1 • 0x06: SHA224 • 0x07: SHA512_224 • 0x08: SHA512_256 START/UPDATE/FINISH/ONE-PASS 模式应设置。仅当 PSS 填充且消息类型(msg_type)是摘要时, 需要填充该字段 (algorithm 字段需要赋值为和计算消息摘要的哈希算法一致), 其它情况下忽略该值。
16-18	reserved1	-
19-19	msg_type	数据类型, 表示 msg_addr 地址指向的数据是原始消息还是消息摘要, 仅 onepass 模式支持消息摘要参考 表 24。
20-23	key_handle	用于操作的密钥, 当 key_type 为 0 时有效。START/UPDATE/FINISH/ONE-PASS 模式应设置。
24-35	reserved2	-
36-43	msg_addr	输入消息的地址。UPDATE/ONE-PASS 模式应设置。final 模式为可选。
44-47	msg_size	输入消息的长度。UPDATE/ONE-PASS 模式应设置。final 模式为可选。如果 msg_type 值为 0x01, msg_size 必须是小于等于 64 字节
48-55	sign_addr	签名数据的地址。FINISH/ONE-PASS 模式应设置。
56-59	sign_size	生成的签名缓冲区的长度或待验证的签名值长度。FINISH/ONE-PASS 模式应设置。
60-67	reserved3	-
68-71	salt_size	盐值的长度, 将用于 RSASSA PSS 填充。FINISH/ONE-PASS 模式应设置。
72-79	sign_ctx	签名或验证的上下文地址。START/UPDATE/FINISH 模式应设置。
80-83	sign_ctx_size	上下文的长度。START/UPDATE/FINISH 模式应设置。
84-91	plain_key_addr	明文密钥, key_type 为 1 时有效。签名需要传公钥和私钥, 验签只需要传公钥, 私钥字段被忽略。START/UPDATE/FINISH/ONE-PASS 模式应设置。参考 表 45。

响应:

表 49 结构体 RSP-RSA-SIGN 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	sign_size	输出签名的大小。验证服务中忽略。
8-11	verification	验证结果。0 表示成功, 其他表示失败。签名服务中忽略。

ECC 明文密钥数据结构

表 50 结构体 ecc_key 定义

位置	字段名称	说明
0-3	curve_id	ECC 曲线编号
4-11	privkey	ECC 私钥的存储地址的地址, 长度由 curve_id 对应的曲线类型决定。
12-19	pubkey	ECC 公钥的存储地址的地址, 长度由 curve_id 对应的曲线类型决定。

A.5.5 Ecdsa

ECDSA 签名/验证命令。

表 51 结构体 CMD-ECDSA 定义

位置	字段名称	说明
0-1	cmd_id	0x0601
2-3	cmd_id_inv	cmd_id 取反
4-10	reserved0	-
11-11	key_type	密钥类型。参考 表 21。START/UPDATE/FINISH/ONE-PASS 模式应设置。
12-12	process_mode	处理模式。参考 表 19。
13-13	direction	签名方向。参考 表 22。
14-14	reserved1	-
15-15	algorithm	签名/验证所使用的 HASH 算法。选项： • 0x00: SM3 • 0x01: MD5 • 0x02: SHA256 • 0x03: SHA384 • 0x04: SHA512 • 0x05: SHA1 • 0x06: SHA224 • 0x07: SHA512_224 • 0x08: SHA512_256 • 0x09: SHA3_224

位置	字段名称	说明
		• 0x0a: SHA3_256 • 0x0b: SHA3_384 • 0x0c: SHA3_512 如果 msg_type 值为 0x01,该参数无效
16-18	reserved2	-
19-19	msg_type	数据类型, 表示 msg_addr 地址指向的数据是原始消息还是消息摘要,仅 onepass 模式支持消息摘要参考 表 24。
20-23	key_handle	用于操作的密钥, 当 key_type 为 0 时有效。
24-35	reserved3	-
36-43	msg_addr	输入消息的地址。
44-47	msg_size	输入消息的长度, 如果 msg_type 值为 0x01,msg_size 必须是小于等于 64 字节, 如果摘要长度大于曲线的宽度, 固件会截断摘要数据。
48-55	sign_addr	签名数据的地址。
56-59	sign_size	生成的签名缓冲区的长度或待验证的签名值长度。
60-71	reserved4	-
72-79	sign_ctx	签名或验证的上下文地址。
80-83	sign_ctx_size	上下文的长度。
84-91	plain_key_addr	明文密钥, key_type 为 1 时有效。签名只需要传私钥, 公钥地址设置为 0, 验签只需要传公钥, 私钥地址设置为 0。START/UPDATE/FINISH/ONE-PASS 模式应设置。参考 表 50。

响应:

表 52 结构体 RSP-ECDSA 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。
4-7	sign_size	输出签名的大小。验证时忽略。
8-11	verification	验证结果。0 表示成功, 其他表示失败。签名服务中忽略。

A.6 随机数生成

提供随机数生成相关的命令。

A.6.1 rng

随机数生成命令。

表 53 结构体 CMD-RNG 定义

位置	字段名称	说明
0-1	cmd_id	0x0a01
2-3	cmd_id_inv	cmd_id 取反
4-14	reserved0	-

位置	字段名称	说明
15-15	algorithm	随机数后处理算法。选项： • 0x00: SM4 CTRDRBG • 0x01: AES CTRDRBG
16-47	reserved1	-
48-55	random_data_addr	生成的随机数据地址。
56-59	require_size	生成的随机数据大小。

响应：

表 54 结构体 RSP-RNG 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7 扩展服务

一些额外的功能服务命令。

A.7.1 otp_write

固件中的 OTP 数据写入命令。此命令可用于写入生命周期、UID、控制字段、传感器响应配置、Patch 配置表和系统密钥等数据。此命令只能在 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE 模式下执行。其中 TEST_MODE 和 DEVELOP_MODE 对写区域无限制，可写 OTP Patch 配置表；MANUFACTURE_MODE 不支持写 OTP Patch 配置表，仅支持写 OTP 扩展区域（OTP 中密钥区域之后的区域，如果此区域存在）。

表 55 结构体 CMD-OTP-WRITE 定义

位置	字段名称	说明
0-1	cmd_id	0xff01
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	ehsm_dst_addr	eHSM 内部地址, 数据将写入到该地址。调用者应非常小心该值和写入内容, 因为 OTP 存储只能写入一次。固件不会检查此值的有效性。
20-23	size	要写入的数据的字节大小。
24-31	host_src_addr	host 地址, 待写入 OTP 的数据从该 host 地址读取。

响应：

表 56 结构体 RSP-OTP-WRITE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7.2 otp_read

固件中的 OTP 数据读取命令。此命令可用于读取生命周期、UID、控制字段、传感器响应配置、Patch 配置表和系统密钥等数据。此命令只能在 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE 模式下执行。其中 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE 均支持读取 OTP Patch 配置表。

表 57 结构体 CMD-OTP-READ 定义

位置	字段名称	说明
0-1	cmd_id	0xff02
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	ehsm_src_addr	eHSM 地址, OTP 数据将从该地址读取。
20-23	size	要读取的数据的字节大小。固件不会检查此值的有效性。
24-31	host_dst_addr	host 地址, 读取到的 OTP 数据将被拷贝到 HOST 的地址。

响应:

表 58 结构体 RSP-OTP-READ 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7.3 get_challenge

获取鉴权的挑战值命令。

表 59 结构体 CMD-GET-CHALLENGE 定义

位置	字段名称	说明
0-1	cmd_id	0xff03
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	type	获取挑战的类型。参考 表 25。
13-47	reserved1	-
48-55	addr	host 地址, 用于存储挑战数据。应足以存放 32 字节的随机数据和 16 字节的 UID。

响应:

表 60 结构体 RSP-GET-CHALLENGE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7.4 auth

鉴权命令。

表 61 结构体 CMD-AUTH 定义

位置	字段名称	说明
0-1	cmd_id	0xff04
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	type	之前获取的挑战类型。参考 表 25。
13-13	alg	用于验证的算法。选项： • 0x01: SM3-SM2 • 0x02: SHA256-SECP256R1 • 0x03: SM4-CMAC • 0x04: AES128-CMAC • 0x05: SHA256_RSA
14-23	reserved1	-
24-31	sign_addr	签名值的 host 地址。
32-35	sign_size	签名的字节大小。对于 eHSM/SOC 调试鉴权，算法 SM3-SM2 和 SHA256-SECP256R1 的值应该是 64 字节，对于算法 SHA256-RSA2048，值应该是 256 字节，对于算法 SHA256-RSA3072，值应该是 384 字节。
36-43	pub_addr	用于验证的公钥的 host 地址。仅对于 eHSM/SOC 调试鉴权有效。
44-47	pub_size	签名者公钥的字节大小。仅对于 eHSM/SOC 调试鉴权有效。对于 SM3-SM2 算法，大小应为 65 字节，包括未压缩标志' 0x04'。对于 SHA256-SECP256R1，大小是 64 字节，SHA256-RSA2048 的大小是 320 字节（64 字节 RSA(e)和 256 字节 RSA(n)），对于 SHA256-RSA3072 的大小是 448 字节（64 字节 RSA(e)和 384 字节 RSA(n)）。
48-55	soc_dbg_bitmap_addr	SOC 调试鉴权位图数据的 host 地址。仅对于 SOC 调试鉴权有效。大小为 5 个字。前 4 个字表示每个位是否打开相应的硬件调试端口（为 1 的位表示打开对应端口，为 0 的位表示不改变对应端口状态）。最后一个字只有最低位有效（表示是否启用硬件调试端口的 129 位）。
56-59	soc_dbg_bitmap_size	SOC 调试鉴权位图数据的大小。固定长度（5 个字）。

响应：

表 62 结构体 RSP-AUTH 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。

A.7.5 close_debug

关闭调试命令。

表 63 结构体 CMD-CLOSE-DEBUG 定义

位置	字段名称	说明
0-1	cmd_id	0xff0e
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	type	关闭调试的类型。参考 表 25。
13-15	reserved1	-
16-23	soc_dbg_bitmap_addr	SOC 调试鉴权位图数据的 host 地址。仅对于 SOC 调试鉴权有效。大小为 5 个字。前 4 个字表示每个位是否关闭相应的硬件调试端口（为 1 的位表示关闭对应端口，为 0 的位表示不改变对应端口状态）。最后一个字只有最低位有效（表示是否关闭了硬件调试端口的 129 位）。
24-27	soc_dbg_bitmap_size	SOC 调试鉴权位图数据的大小。固定长度（5 个字）。

响应：

表 64 结构体 RSP-CLOSE-DEBUG 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。

A.7.6 fw_upgrade

固件镜像升级命令。

表 65 结构体 CMD-FW-UPGRADE 定义

位置	字段名称	说明
0-1	cmd_id	0xff05
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	process_mode	处理模式。参考 表 19。
13-31	reserved1	-
32-39	image_addr	镜像地址。ONE-PASS 时，它是整个镜像的 host 地址，START 时为镜像头部，UPDATE 时为镜像加密部分地址，或者 FINISH 时为镜像最后一部分加密内容（可能为空）。
40-43	image_size	升级镜像的大小（以字节为单位）。
44-51	storage_addr	输出的启动镜像的 host 地址。

此命令输入固件升级镜像，输出固件启动镜像。

响应：

表 66 结构体 RSP-FW-UPGRADE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.7.7 soc_verify

验证 SOC 启动镜像命令。

表 67 结构体 CMD-SOC-VERIFY 定义

位置	字段名称	说明
0-1	cmd_id	0xff06
2-3	cmd_id_inv	cmd_id 取反
4-13	reserved0	-
14-14	check_version	是否检查版本计数器。选项： • 0x00: 不检查版本计数器 • 0x01: 需要检查版本计数器 • 其他值为保留
15-15	reserved1	-
16-23	image_addr	镜像的 host 地址。
24-27	image_size	镜像的大小（以字节为单位）。
28-35	image_out_addr	解密后的镜像（或如果镜像为明文则进行复制）存储的 host 地址。可以指向原始镜像地址。

此命令用于校验 SOC 启动镜像并将解密后的镜像输出到 host 地址。

响应：

表 68 结构体 RSP-SOC-VERIFY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.7.8 install_random_key

安装随机密钥命令。

表 69 结构体 CMD-INSTALL-RANDOM-KEY 定义

位置	字段名称	说明
0-1	cmd_id	0xff08
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	key_level	安装的 OTP 密钥级别 选项： • 0x01: 随机密钥使用 CHIP_ROOT_KEY 加密。 • 0x02: 随机密钥使用 DEVICE_ROOT_KEY 加密。

位置	字段名称	说明
13-13	key_type	密钥类型。选项： • 0x01: 用于对称密钥 • 0x02: 用于 SM2 私钥 • 0x04: 用于 secp256r1 私钥 对于对称密钥，输出是随机的 32 字节密钥。对于 SM2 或 SECP256R1，输出是随机生成的有效私钥。
14-15	key_slot_id	指示密钥安装到 eHSM 的哪个硬件槽位 ID（OTP 中的密钥索引），可用的 ID 范围请参考 OTP 密钥章节。
16-16	key_last_flag	指示安装的密钥是否为最后一个密钥。选项： • 0x00: 安装的密钥不是最后一个密钥。 • 0x01: 安装的密钥是最后一个密钥。
17-19	reserved1	-

仅支持在 TEST_MODE/DEVELOP_MODE/MANUFACTURE_MODE 下执行（MANU 模式下仅支持密钥级别为 2 级的密钥安装）。OTP 属性的前三个密钥必须是固定的。请参考硬件 TRM 文档。

响应：

表 70 结构体 RSP-INSTALL-RANDOM-KEY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.7.9 install_encrypt_key

安装加密密钥命令。

表 71 结构体 CMD-INSTALL-ENCRYPT-KEY 定义

位置	字段名称	说明
0-1	cmd_id	0xff09
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	key_level	eHSM 或 SOC 密钥级别。选项： • 0x01: 加密密钥使用 CHIP_ROOT_KEY 加密。 • 0x02: 加密密钥使用 DEVICE_ROOT_KEY 加密。
13-13	key_type	密钥类型。选项： • 0x01: 用于对称密钥 • 0x02: 用于 SM2/SECP256R1 私钥 • 0x04: 用于公钥的哈希值 对于对称密钥，输入是随机的 32 字节密钥。对于 SM2 或 SECP256R1，输入是随机生成的有效私钥。对于哈希公钥，输入是公钥的哈希值。
14-15	key_slot_id	指示密钥安装到 eHSM 的哪个硬件槽位 ID（OTP 中的密钥索引），可用的 ID 范围请参考 OTP 密钥章节，最小为 3。

位置	字段名称	说明
16-16	key_last_flag	指示安装的密钥是否为最后一个密钥。选项： 0x00: 安装的密钥不是最后一个密钥。 0x01: 安装的密钥是最后一个密钥。
17-35	reserved1	-
36-43	input_addr	输入密钥或 HASH 值的 host 地址。32 字节密钥（或哈希）+ 4 字节密钥（或哈希）的 CRC 值，用 0x00 填充至 48 字节，并使用 RTL（eHSM/SOC）KEK 加密的结果。
44-47	input_size	输入密钥或 HASH 值的大小，长度固定为 48 字节。

仅支持在 TEST_MODE/DEVELOP_MODE/MANUFACTURE_MODE 下执行。此命令不支持安装 CHIP_ROOT_KEY，也不支持在 MANU 模式下安装密钥级别为 1 的固件。如果用于安装 DEV_ROOT_KEY 或 USER_ROOT_KEY，则其属性必须固定。请参考硬件 TRM 文档。

响应：

表 72 结构体 RSP-INSTALL-ENCRYPT-KEY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

用于获取 eHSM EMU 状态信息的数据结构。

表 73 结构体 ehsm_emu_status 定义

位置	字段名称	说明
0-7	o_hsm_status	eHSM 状态
8-11	o_hsm_err_sensor	传感器检测到的错误
12-19	o_hsm_err_hw	硬件检测到的错误
20-27	o_hsm_err_fw	固件检测到的错误

A.7.10 get_emu

获取 EMU 状态命令。

表 74 结构体 CMD-GET-EMU 定义

位置	字段名称	说明
0-1	cmd_id	0x0070
2-3	cmd_id_inv	cmd_id 取反
4-39	reserved0	-
40-47	emu_addr	EMU 状态的输出缓冲区。参考 表 73 。
48-51	emu_size	emu_addr 的缓冲区大小。

响应：

表 75 结构体 RSP-GET-EMU 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	emu_size	EMU 状态的输出大小. 仅在命令成功时有效.

A.7.11 change_lifecycle

更改生命周期状态命令。

表 76 结构体 CMD-CHANGE-LIFECYCLE 定义

位置	字段名称	说明
0-1	cmd_id	0x0f01
2-3	cmd_id_inv	cmd_id 取反
4-7	type	生命周期类型。选项： • 0x01: 更改为测试模式 (TEST_MODE) • 0x02: 更改为开发模式 (DEVELOP_MODE) • 0x03: 更改为制造模式 (MANUFACTURE_MODE) • 0x04: 更改为用户模式 (USER_MODE) • 0x05: 更改为调试模式 (DEBUG_MODE) • 0x06: 更改为销毁模式 (DESTORY_MODE) • 其他值保留

响应:

表 77 结构体 RSP-CHANGE-LIFECYCLE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.7.12 change_control_field

更改控制字段命令。

表 78 结构体 CMD-CHANGE-CONTROL-FIELD 定义

位置	字段名称	说明
0-1	cmd_id	0x0f02
2-3	cmd_id_inv	cmd_id 取反
4-4	type	控制字段类型。选项： • 0x00: 硬件 (HW) 控制字段 • 0x01: EHSM 控制字段 • 0x02: SOC 控制字段

位置	字段名称	说明
		其他值保留
5-5	reserved0	-
6-7	size	值的大小, 此值固定为 8, 即 2 个 word, 请参考 TRM 文档。
8-15	value	要更新的值的地址。

响应:

表 79 结构体 RSP-CHANGE-CONTROL-FIELD 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7.13 set_uart_baudrate

设置 UART 波特率命令。

表 80 结构体 CMD-SET-UART-BAUDRATE 定义

位置	字段名称	说明
0-1	cmd_id	0x0f04
2-3	cmd_id_inv	cmd_id 取反
4-7	baud_div	UART 的分频值, 指 CPU 频率值除以 UART 波特率值所得的结果。

响应:

表 81 结构体 RSP-SET-UART-BAUDRATE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7.14 fw_reg_wr

CFG 数据写入命令。

表 82 结构体 CMD-FW-REG-WR 定义

位置	字段名称	说明
0-1	cmd_id	0xff23
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	ehsm_dst_addr	eHSM 地址, 数据将写入此地址。
20-23	size	要写入的数据的字节大小。
24-31	host_src_addr	host 地址, 数据将从此地址读取。

CFG 的地址范围请参考硬件 TRM 手册。

响应:

表 83 结构体 RSP-FW-REG-WR 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.7.15 fw_reg_rd

CFG 数据读取命令。

表 84 结构体 CMD-FW-REG-RD 定义

位置	字段名称	说明
0-1	cmd_id	0xff24
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	ehsm_src_addr	eHSM 地址, 数据将从此地址读取。
20-23	size	要读取的数据的字节大小。固件无需验证此值的有效性。
24-31	host_dst_addr	host 地址, 数据将写入此地址。缓冲区应具有足够的大小。

CFG 的地址范围请参考硬件 TRM 手册。

响应:

表 85 结构体 RSP-FW-REG-RD 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

用于获取 eHSM 版本信息的数据结构

表 86 结构体 ehsm_version 定义

位置	字段名称	说明
0-0	type	固件类型 选项: 0: BootLoader 1: Firmware
1-1	ver_major	主版本号。
2-2	ver_minor	次版本号。
3-3	ver_patch	补丁版本号。
4-11	ver_pre_release	预发布版本字符串, 例如 <code>alpha.1</code> , <code>rc3</code> 。对于正式版本, 这是一个空字符串。
12-19	reserved0	-
20-23	pke_engine_ver	PKE 引擎版本号。
24-27	pke_lib_ver	PKE 库版本号。格式: 0x23080301 表示 2023-08-03 版本 1。
28-31	ske_engine_ver	SKE 引擎版本号。

位置	字段名称	说明
32-35	ske_lib_ver	SKE 库版本号。格式: 同 pke_lib_ver。
36-39	hash_engine_ver	HASH 引擎版本号。
40-43	hash_lib_ver	HASH 库版本号。格式: 同 pke_lib_ver。
44-47	trng_engine_ver	TRNG 引擎版本号。
48-51	trng_lib_ver	TRNG 库版本号。格式: 同 pke_lib_ver。
52-55	hw_ver	硬件版本号, 参见 OSR_eHSM_HP_Technical_Reference_Manual_xxx.pdf 中的 SYS_VER1。
56-71	uid	从 OTP 读取的 16 字节 UID 值。
72-127	reserved1	-

A.7.16 ehsm_read_ver

读取固件版本命令。

表 87 结构体 CMD-EHSM-READ-VER 定义

位置	字段名称	说明
0-1	cmd_id	0xff10
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	ver_addr	版本信息的 host 地址。参考 表 86。
20-23	ver_size	版本缓冲区的大小

响应:

表 88 结构体 RSP-EHSM-READ-VER 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。

A.7.17 enter_wfi

进入 WFI (节能) 模式命令。

表 89 结构体 CMD-ENTER-WFI 定义

位置	字段名称	说明
0-1	cmd_id	0xff26
2-3	cmd_id_inv	cmd_id 取反

eHSM 在收到命令后, 当所有 Mailbox channel 上都没有命令时, 会进入 WFI 状态, 直到下一次被 Mailbox 或其它中断唤醒。Host 应当在发送此命令后不再发送任何命令, 直到硬件收到 eHSM 的 cpu_wfi 状态信号。

响应:

表 90 结构体 RSP-ENTER-WFI 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.8 密钥管理

密钥管理相关的命令, 包含密钥的生成、导入、导出、删除等。

A.8.1 ehsm_gen_key

生成对称和非对称密钥命令。

表 91 结构体 CMD-EHSM-GEN-KEY 定义

位置	字段名称	说明
0-1	cmd_id	0x080d
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	algo_id	生成密钥的算法 ID。参考 表 26。
13-15	reserved1	-
16-17	rsa_e_bit_size	RSA 公钥 e 的位长度。最大为 64。当公钥的位长度为 17 时, 输出的公钥 e 值固定为 65537 (0x10001)。仅对生成 RSA 密钥有效。
18-19	key_size	创建密钥的长度。仅对生成 HMAC 密钥有效, 最大长度为 512 字节。
20-23	key_handle	生成密钥的密钥句柄, 设置为 0xFFFFFFFF 表示由密钥管理服务分配的句柄。
24-27	permit	生成的新密钥的权限值。密钥可以有多个权限。参考 表 27。
28-31	reserved2	-
32-39	dh_common_addr	DH 公共数据的地址。其数据格式为{p_len + p_value + q_len + q_value + g_len + g_value + h_len + h_value}。x_len 的值表示 x_value 值的长度。h_value 是可选的, 将 h_len 的值设置为 0 表示 h_value 不存在。x_len 的长度为 4 字节。仅对生成 DH 密钥有效。
40-43	dh_common_size	DH 公共数据的长度。仅对生成 DH 密钥有效, 最大长度为 2064 字节。

响应:

表 92 结构体 RSP-EHSM-GEN-KEY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	key_handle	返回的密钥句柄。

A.8.2 ehsm_key_derive

密钥派生命令。

表 93 结构体 CMD-EHSM-KEY-DERIVE 定义

位置	字段名称	说明
0-1	cmd_id	0x080e
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	key_deriv_func	密钥派生的函数标识符。选项： 0x01: KDFX963 算法 0x02: PBKDF2 算法
13-13	reserved1	-
14-14	derive_type	密钥派生类型。选项： 0x01: 使用用户密码派生密钥 0x02: 使用用户父密钥派生密钥
15-15	hash_alg	密钥派生的哈希算法。选项： 0x00: SM3 算法 0x01: MD5 算法 0x02: SHA-256 算法 0x03: SHA-384 算法 0x04: SHA-512 算法 0x05: SHA-1 算法 0x06: SHA-224 算法 0x07: SHA-512/224 算法 0x08: SHA-512/256 算法 0x09: SHA3-224 算法 0x0a: SHA3-256 算法 0x0b: SHA3-384 算法 0x0c: SHA3-512 算法
16-16	algo_id	派生密钥的密钥算法 ID。注意：仅对称密钥算法 ID 有效。参考 表 26。
17-17	key_out_dir	密钥输出方向，当前仅支持配置成 0x01。选项： 0x01: 密钥将存储在 eHSM KMS 中
18-19	reserved2	-
20-23	key_handle	派生密钥的密钥句柄，如果已存在则覆盖。0xFFFFFFFF 值表示自动分配句柄。
24-27	permit	派生密钥的权限值。密钥可以有多个权限位组合。参考 表 27。
28-31	reserved3	-
32-35	parent_key_handle	密钥派生的父密钥，当 derive_type 为 KMS_KEY_DERIVE_USER_PASSWORD 时忽略
36-43	salt_data_addr	盐数据的 host 地址。
44-47	salt_data_size	派生密钥的盐参数。最大长度为 128 字节。

位置	字段名称	说明
48-55	pw_data_addr	密码数据的 host 地址。当 derive_type 为 KMS_KEY_DERIVE_USER_KEYHANDLE 时忽略。
56-59	pw_data_size	派生密钥的密码参数。最大长度为 128 字节。
60-79	reserved4	-
80-83	itera_times	迭代次数，仅对 PBKDF2 有效，过低的迭代次数影响安全性。
84-91	reserved5	-
92-93	key_size	派生密钥的字节大小。如果算法 ID 是 HMAC 密钥，则最大大小为 512 字节。其他算法 ID 必须设置为与算法要求相匹配。

响应：

表 94 结构体 RSP-EHSM-KEY-DERIVE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功，其它值表示失败。
4-7	key_handle	返回的密钥句柄。

SM2 参数结构体。

表 95 结构体 sm2_param_struct 定义

位置	字段名称	说明
0-0	sm2_role	密钥交换的 SM2 角色。选项： 0x00: KMS_SM2_ROLE_SPONSOR 0x01: KMS_SM2_ROLE_RESPONSOR
1-3	reserved0	-
4-7	local_tmp_key_handle	SM2 交换本地临时密钥的密钥句柄。
8-15	s1_s2_value_addr	SM2 交换 s1_s2 值的 host 地址。
16-23	sa_sb_value_addr	SM2 交换 sa_sb 值的 host 地址。
24-31	peer_temp_pubkey_addr	SM2 交换对方临时公钥值的 host 地址。
32-35	peer_temp_pubkey_size	对方临时公钥的长度。

A.8.3 ehsm_key_exchange

密钥交换命令。

表 96 结构体 CMD-EHSM-KEY-EXCHANGE 定义

位置	字段名称	说明
0-1	cmd_id	0x080f
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-12	algo_id	密钥交换生成的密钥算法 ID，仅对称密钥算法 ID 有效。参考 表 26。
13-13	reserved1	-

位置	字段名称	说明
14-17	reserved2	-
18-19	key_size	生成的密钥的字节大小。如果算法 ID 是 HMAC 密钥, 则最大大小为 512 字节。其他算法 ID 必须设置为与算法要求相匹配的大小。
20-23	key_handle	交换密钥的密钥句柄, 如果已存在则会被覆盖。0xFFFFFFFF 值表示自动分配的句柄。
24-27	permit	交换密钥的权限值。密钥可以由多个权限组合而成。参考 表 27。
28-31	local_key_handle	内部私钥的密钥句柄。
32-39	dh_common_addr	DH 公共数据的地址。其数据格式为{p_len + p_value + q_len + q_value + g_len + g_value + h_len + h_value}。x_len 的值表示 x_value 的值的长度。h_value 是可选的, 将 h_len 的值设置为 0 表示 h_value 不存在。x_len 的长度为 4 字节。仅对生成 DH 密钥有效。
40-43	dh_common_size	DH 公共数据的长度。仅对生成 DH 密钥有效。最大大小为 2064 字节。
44-51	remote_pubkey_data	远程公钥数据的 host 地址。
52-55	remote_pubkey_size	远程公钥的大小。最大大小为 512 字节。
56-67	reserved3	-
68-75	sm2_param_addr	指向 SM2 参数的 host 地址。参考 表 95。

响应:

表 97 结构体 RSP-EHSM-KEY-EXCHANGE 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。
4-7	key_handle	返回的密钥句柄。

A.8.4 ehsm_import_key

密钥导入命令。

表 98 结构体 CMD-EHSM-IMPORT-KEY 定义

位置	字段名称	说明
0-1	cmd_id	0x0801
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	reserved1	-
20-23	key_handle	导入密钥的密钥句柄, 如果已存在则会被覆盖。0xFFFFFFFF 值表示自动分配句柄。
24-27	transport_key_handle	用于传输保护(加密)导入密钥的密钥句柄, 如果为 0xFFFFFFFF, 表示明文导入。
28-31	reserved2	-
32-35	authenticity_key_handle	用于传输保护(验证)导入密钥的密钥句柄, 如果为 0xFFFFFFFF, 表示不校验 MAC。
36-43	key_data_addr	导入密钥数据的地址。不同密钥的格式不同。

位置	字段名称	说明
44-47	key_data_size	导入密钥的长度。最大长度为 2348 字节。
48-55	key_signature_addr	导入密钥签名数据的地址。
56-59	key_signature_size	导入密钥签名数据的长度。最大长度为 32 字节。

响应:

表 99 结构体 RSP-EHSM-IMPORT-KEY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败。
4-7	key_handle	返回的密钥句柄。

A.8.5 ehsm_export_key

密钥导出命令

表 100 结构体 CMD-EHSM-EXPORT-KEY 定义

位置	字段名称	说明
0-1	cmd_id	0x0802
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-15	reserved1	-
16-16	req_key_part	要导出的密钥数据的部分。选项: • 0x01: 公钥部分 • 0x02: 私钥部分, 或对称密钥 • 0x03: 密钥对
17-19	reserved2	-
20-23	key_handle	要导出的密钥的密钥句柄。
24-27	transport_key_handle	用于传输保护(解密)导出密钥的密钥句柄, 如果为 0xFFFFFFFF, 表示明文导出密钥。
28-31	reserved3	-
32-35	authenticity_key_handle	用于传输保护(签名)导出密钥的密钥句柄, 如果为 0xFFFFFFFF, 表示不生成 MAC。
36-43	key_data_addr	存储导出密钥数据的 host 地址。不同密钥的格式和长度不同。
44-47	key_data_size	导出密钥数据缓冲的长度。
48-55	key_signature_addr	存储导出密钥签名数据的 host 地址。
56-59	key_signature_size	导出密钥签名数据的长度。

导出与密钥句柄对应的固件密钥数据。

响应:

表 101 结构体 RSP-EHSM-EXPORT-KEY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	key_data_size	返回的导出密钥数据的实际长度。
8-11	key_signature_size	返回的导出密钥签名数据 (MAC) 的实际长度。

A.8.6 ehsm_remove_key

移除密钥命令。

表 102 结构体 CMD-EHSM-REMOVE-KEY 定义

位置	字段名称	说明
0-1	cmd_id	0x0806
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-15	key_handle	要移除的密钥的密钥句柄。

密钥是否移除成功受权限配置影响。

响应:

表 103 结构体 RSP-EHSM-REMOVE-KEY 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.

A.8.7 ehsm_get_pub_from_priv

从私钥计算公钥命令。

表 104 结构体 CMD-EHSM-GET-PUB-FROM-PRIV 定义

位置	字段名称	说明
0-1	cmd_id	0x0805
2-3	cmd_id_inv	cmd_id 取反
4-11	reserved0	-
12-19	reserved1	-
20-23	key_handle	私钥的句柄。
24-35	reserved2	-
36-43	dh_common_addr	DH 公共数据的地址。其数据格式为 {p_len + p_value + q_len + q_value + g_len + g_value + h_len + h_value}。x_len 的值表示 x_value 的长度。h_value 是可选的, 将 h_len 的值设置为 0 表示 h_value 不存在。x_len 的长度为 4 字节。仅对生成 DH 密钥有效。

位置	字段名称	说明
44-47	dh_common_size	DH 参数 p/q/g 数据的长度。仅对生成 DH 密钥有效。最大长度为 2064 字节。
48-55	public_key_addr	存储公钥数据的 host 地址。
56-59	public_key_buffer_size	公钥数据的缓冲区长度

响应:

表 105 结构体 RSP-EHSM-GET-PUB-FROM-PRIV 定义

位置	字段名称	说明
0-3	ret_code	0x0000A55AU 表示操作成功, 其它值表示失败.
4-7	algo_id	返回的公钥算法 ID。
8-11	public_key_size	返回的公钥实际大小。

B 错误码

下表定义了 Mailbox 命令返回的错误码编码。

表 106 错误码定义

Name	Value
EHSM_ERR_PARAM_ERROR	1
EHSM_ERR_QUEUE_FULL	2
EHSM_ERR_NOT_INIT	3
EHSM_ERR_REPEATED_INIT	4
EHSM_ERR_FREE_REPEATED	5
EHSM_ERR_QUEUE_EMPTY	6
EHSM_ERR_CMDPOOL_EMPTY	7
EHSM_ERR_INVALID_ADDRESS	8
EHSM_ERR_WONG_IRQ_NUM	9
EHSM_ERR_NOT_SUPPORT	10
EHSM_ERR_INVALID_HANDLE	11
EHSM_ERR_MISMATCH_KEY_USAGE	12
EHSM_ERR_NOT_ALIGNED	13
EHSM_ERR_DATA_NOT_EMPTY	14
EHSM_ERR_INIT_DEV_FAIL	15
EHSM_ERR_WRITE_DEV_FAIL	16
EHSM_ERR_READ_DEV_FAIL	17
EHSM_ERR_ERASE_DEV_FAIL	18
EHSM_ERR_CTX_OVERFLOW	19
EHSM_ERR_OUTPUT_OVERFLOW	20
EHSM_ERR_HMAC_K_BUF_OCCUPIED	21
EHSM_ERR_HMAC_INVALID_K_IDX	22
EHSM_ERR_REMAP_FAILED	23
EHSM_ERR_REMAP_ADDRESS_OVERFLOW	24
EHSM_ERR_MAC_LEN_WRONG_FORMAT	25
EHSM_ERR_MAC_VRY_FAILED	26
EHSM_ERR_INVALID_DIR	27
EHSM_ERR_OUT_OF_MEM	28
EHSM_ERR_DBG_BUF_OVERFLOW	29
EHSM_ERR_DBG_CMD_INCOMPLETE	30
EHSM_ERR_DBG_PROTO_ERROR	31
EHSM_ERR_INVALID_CHANNEL	32
EHSM_ERR_NO_CMD_EXEC_PERM	33
EHSM_ERR_INVALID_CMD	34

Name	Value
EHSM_ERR_SM2_DATA_BUF_OVERFLOW	35
EHSM_ERR_SM2_SIGNATURE_GEN_FAILED	36
EHSM_ERR_SM2_SIGNATURE_VRY_FAILED	37
EHSM_ERR_RSA_CALCULATE_FAILED	38
EHSM_ERR_WRONG_SZ_OF_SIGNATURE	39
EHSM_ERR_LIMIT_OF_AUTHORITY	40
EHSM_ERR_VERIFY_KEY_INTEGRITY	43
EHSM_ERR_EXEC_CRYPTO_LIB	44
EHSM_ERR_MISMATCH_KEY_PERMISSION	45
EHSM_ERR_NOT_EXIST_KEY_PART	46
EHSM_ERR_NOT_EXIST_KEY	47
EHSM_ERR_INVALID_KEY_ID	48
EHSM_ERR_INVALID_ALGORITHM	49
EHSM_ERR_DH_KEY_TOO_LEN	50
EHSM_ERR_NOT_ALLOWED_CREATION_KEY	51
EHSM_ERR_NOT_EMPTY_KEY_HANDLE	52
EHSM_ERR_IV_OVERFLOW	53
EHSM_ERR_KEY_SIGNATURE_SZ_TOO_SHORT	54
EHSM_ERR_XTS_WRONG_DATA_LENGTH	55
EHSM_ERR_INVALID_CIPHER_MODE	57
EHSM_ERR_INVALID_PADDING	58
EHSM_ERR_NOT_FREE_SPACE	59
EHSM_ERR_DATA_CHECKSUM	60
EHSM_ERR_NVM_WRITE_DATA	61
EHSM_ERR_NVM_SYNC_DATA	62
EHSM_ERR_ECDSA_SIGNATURE_GEN_FAILED	63
EHSM_ERR_ECDSA_SIGNATURE_VRY_FAILED	64
EHSM_ERR_SM2_CIPHER_ENC_FAILED	65
EHSM_ERR_SM2_CIPHER_DEC_FAILED	66
EHSM_ERR_INPUT_OVERFLOW	67
EHSM_ERR_RSA_SIGNATURE_GEN_FAILED	68
EHSM_ERR_RSA_SIGNATURE_VRY_FAILED	69
EHSM_ERR_KEY_NOT_FOUND	70
EHSM_ERR_WRONG_IV_SIZE	71
EHSM_ERR_NVM_ENCRYPTION_KEY_DATA	72
EHSM_ERR_NVM_SIGNATURE_KEY_DATA	73
EHSM_ERR_GCM_TAG_VRY_FAILED	74
EHSM_ERR_CCM_TAG_VRY_FAILED	75
EHSM_ERR_KEY_UPDATE_ERROR	76
EHSM_ERR_KEY_INVALID	77

Name	Value
EHSM_ERR_KEY_NOT_AVAILABLE	78
EHSM_ERR_KEY_EMPTY_ERROR	79
EHSM_ERR_KEY_WRITE_PROTECTED	80
EHSM_ERR_EHSM_LIFECYCLE_LIMIT	81
EHSM_ERR_WRONG_CHALLENGE_TYPE	82
EHSM_ERR_WRONG_DATA_LENGTH	83
EHSM_ERR_WRONG_ALGORITHM	84
EHSM_ERR_WRONG_KEY_TYPE	85
EHSM_ERR_DEBUG_AUTH_PK_HASH_MISMATCH	86
EHSM_ERR_DEBUG_AUTH_FAILED	87
EHSM_ERR_SEQUENCE	88
EHSM_ERR_NO_DATA_IN_MAC_FINAL	89
EHSM_ERR_WRONG_KEY_LEVEL	90
EHSM_ERR_OTP_KEY_INSTALL_TWICE	91
EHSM_ERR_OTP_WRITE_CMP_ERROR	92
EHSM_ERR_REMOVE_USER_KEY_FAILED	93
EHSM_ERR_UTC_REG_VALUE_MISMATCH	94
EHSM_ERR_UTC_RESYNC_FAILED	95
EHSM_ERR_WONG_UTC_TO_IDX	96
EHSM_ERR_COUNTER_REG_VALUE_MISMATCH	97
EHSM_ERR_UTC_TIMER_NOT_SYNC	98
EHSM_ERR_UTC_TIMER_DISABLE	99
EHSM_ERR_ALL_COUNTER_OCCURRED	100
EHSM_ERR_INVALID_COUNTER_ID	101
EHSM_ERR_COUNTER_DISABLE	102
EHSM_ERR_COUNTER_OVERFLOW	103
EHSM_ERR_UTC_TIMER_OVERFLOW	104
EHSM_ERR_INVALID_DH_PQGH_SIZE	105
EHSM_ERR_DBG_PROCESSING	106
EHSM_ERR_DH_PARAM_OF_PQ_IS_EVEN	107
EHSM_ERR_WRONG_VERSION_COUNTER	108
EHSM_ERR_INVALID_CODE_FLAG	109
EHSM_ERR_WRONG_PUBKEY	110
EHSM_ERR_FW_VERIFY_FAILED	111
EHSM_ERR_WRONG_FW_TYPE	112
EHSM_ERR_STL_FAILED	113
EHSM_ERR_WRONG_PROC_MODE	114
EHSM_ERR_DATA_CHECK_ERROR	115
EHSM_ERR_WRONG_TAG_SIZE	116
EHSM_ERR_WRONG_NONCE_SIZE	117

Name	Value
EHSM_ERR_WRONG_AAD_SIZE	118

C 字符调试鉴权协议

字符调试鉴权协议所有数据使用 ASCII 字符串进行传输，可以通过串口和 Mailbox 通道传输，实现调试鉴权的功能主要包含以下几条指令：

- 获取挑战值指令
- 签名校验指令
- 关闭调试指令
- 测试指令

以下指令格式中 type 代表调试类型，其值可以为 0x31 或 0x33, 0x31 代表 eHSM 调试鉴权，0x33 代表 SOC 调试鉴权，alg 代表算法类型，其值可以为 0x31、0x32、0x33、0x34 或 0x35, 0x31 代表 SM3-SM2, 0x32 代表 SHA256-SECP256R1, 0x33 代表 SM4-CMAC, 0x34 代表 AES128-CMAC, 0x35 代表 RSA2048-SHA256 或者 RSA3072-SHA256。

RSA 签名算法填充模式为 RSA-PSS 模式，RSA2048-SHA256 对应的公钥为：64 字节 e 值+256 字节 n 值，签名长度为 256 字节，RSA3072-SHA256 对应的公钥为：64 字节 e 值+384 字节 n 值，签名长度为 384 字节。

C.1 获取挑战值指令

请求数据格式：“REQ” + type + alg + “@”

例如：字符串“REQ11@”代表 eHSM 调试鉴权，鉴权算法为 SM3-SM2

成功响应格式：“CHA” + type + “:” + challeng in char + “@”

challeng in char：挑战值转换的 16 进制字符串，挑战值包含 32 字节随机数+16 字节 UID

例如响应数据为：

```
1| CHA3:6343F25AA6AA8E5705CBD3D6B19BEE50D4495575562B5B1F6AE415066E2228C3008E69D927E
2| 34EB857F7BD4FAE47F9820
```

代表当前为 SOC 调试鉴权，对应的 32 字节随机数为[0x63, 0x43, 0xF2, 0x5A, 0xA6, 0xAA, 0x8E, 0x57, 0x05, 0xCB, 0xD3, 0xD6, 0xB1, 0x9B, 0xEE, 0x50, 0xD4, 0x49, 0x55, 0x75, 0x56, 0x2B, 0x5B, 0x1F, 0x6A, 0xE4, 0x15, 0x06, 0x6E, 0x22, 0x28, 0xC3], 16 字节的 UID 为 [0x00, 0x8E, 0x69, 0xD9, 0x27, 0xE3, 0x4E, 0xB8, 0x57, 0xF7, 0xBD, 0x4F, 0xAE, 0x47, 0xF9, 0x82]

失败响应格式：“CHA” + type + “:get random fail” + “@”

C.2 签名校验指令

请求数据格式：“RSP” + type + alg + “:” + publickey + “:” + signature + “@” publickey: 非对称算法的公钥的 16 进制字符串，signature: 签名值的 16 进制字符串

例如：


```
1| RSP11:09F9DF311E5421A150DD7D161E4BC5C672179FAD1833FC076BB08FF356F35020CCEA490CE2
2| 6775A52DC6EA718CC1AA600AED05FBF35E084A6632F6072DA9AD13:A8E776DCF22DE9C5E847CC321
3| 5A298D1B99884D68A43A83A8C236347287514A2B7AA77A70727BF37A3BE69982F87778AA28C72183
4| 2C64BE5139646948EA2AF6B0
```

代表 SOC 调试鉴权的签名检验指令，算法类型是 SM3-SM2，SM2 的公钥为 [0x09, 0xF9, 0xDF, 0x31, 0x1E, 0x54, 0x21, 0xA1, 0x50, 0xDD, 0x7D, 0x16, 0x1E, 0x4B, 0xC5, 0xC6, 0x72, 0x17, 0x9F, 0xAD, 0x18, 0x33, 0xFC, 0x07, 0x6B, 0xB0, 0x8F, 0xF3, 0x56, 0xF3, 0x50, 0x20, 0xCC, 0xEA, 0x49, 0x0C, 0xE2, 0x67, 0x75, 0xA5, 0x2D, 0xC6, 0xEA, 0x71, 0x8C, 0xC1, 0xAA, 0x60, 0x0A, 0xED, 0x05, 0xFB, 0xF3, 0x5E, 0x08, 0x4A, 0x66, 0x32, 0xF6, 0x07, 0x2D, 0xA9, 0xAD, 0x13], 签名值为 [0xA8, 0xE7, 0x76, 0xDC, 0xF2, 0x2D, 0xE9, 0xC5, 0xE8, 0x47, 0xCC, 0x32, 0x15, 0xA2, 0x98, 0xD1, 0xB9, 0x98, 0x84, 0xD6, 0x8A, 0x43, 0xA8, 0x3A, 0x8C, 0x23, 0x63, 0x47, 0x28, 0x75, 0x14, 0xA2, 0xB7, 0xAA, 0x77, 0xA7, 0x07, 0x27, 0xBF, 0x37, 0xA3, 0xBE, 0x69, 0x98, 0x2F, 0x87, 0x77, 0x8A, 0xA2, 0x8C, 0x72, 0x18, 0x32, 0xC6, 0x4B, 0xE5, 0x13, 0x96, 0x46, 0x94, 0x8E, 0xA2, 0xAF, 0x6B]

响应数据格式：

- 校验成功的响应数据：“ENDPASS@”
- 公钥校验失败的响应数据：“END_PUBKEY_FAIL@”
- 签名校验指令数据错误的响应数据：“END_DATA_FAIL@”
- 签名校验失败的响应数据：“END_VERIFY_FAIL@”
- 参数错误的响应数据：“END_PARAMETER_FAIL@”

C.3 关闭调试指令

调试鉴权成功后可通过该指令关闭调试

请求数据格式：“DIS” + type + “@”

响应数据格式：

- 关闭成功的响应数据：“ENDPASS@”
- 关闭失败的响应数据：“END_PARAMETER_FAIL@”

C.4 测试指令

该指令用于测试，例如请求数据为“TESTabcd@”，响应数据为“TESTabcd”

参考文献

- [1] O. IP, 《OSR Standard eHSM 技术参考手册》.
- [2] OSR, 《OSR eHSM 外部依赖 API 接口参考手册》.